Notes for C++

Section 0 : Introduction

Who should do the course?

Students who want to major in any one (or two) of Computer Science, Data Science, Information Technology, Information Systems, Software Engineering, Games Programming (minor).

What do I need to know before I start the course?

No experience is required. Students with considerable programming experience in C++, Java, Python or a similar language can apply to skip the course. For more information, contact the Academic Dean Information Sciences. Note that skipping a course does not give you credit for the course. It simply means that you room in your degree to fit in a different course.

What is an Algorithm?

An algorithm is a series of instructions (and may have nothing to do with a computer).
Algorithms have been used for centuries to provide precise, detailed instructions.
Algorithms are written (or spoken) in *natural languages*, e.g. English is a natural language.

Here is an example of an algorithm (nothing to do with a computer):

1. Preheat the oven to 375 F
2. Mix the flour, baking powder, salt and sugar in a large bowl
3. Add the egg, milk and butter, stirring only enough to dampen the flour – the batter should not be smooth.
4. Stir 1 cup of blueberries into the batter.
5. Spoon into muffin pans, filling each cup about two-thirds full.
6. Bake for about 20 to 25 minutes.

Are these instructions precise enough? What could go wrong? For some ideas on the difficulties of getting an algorithm correct, have a look at https://www.youtube.com/watch?v=cDA3_5982h8

What is a Computer Program?

A computer program is a series of instructions written in a particular *programming language*. A program consists of instructions (statements) written in a specific form that a computer is able to translate and execute. Execute means to carry out the instructions.

Programming Languages

There are about 700 programming languages. Different languages were created to do different types of programming tasks. For example, the Structured Query Language (SQL) was developed to manage data held in a database. A language provides rules for writing instructions. In this course we use the language C++ which forms the basis of a family of related languages including C, C#, C++ and Java. (Interesting fact – there are 700 programming languages but there are over 6,500 spoken languages.)

Syntax

All languages including ***programming languages*** (such as C++) and ***natural languages*** (such as English) have rules on how to properly construct sentences (or equivalent) in the language. These rules are known as the **syntax** of the language. Two examples of syntax rules for English are (a) start every sentence with a capital letter, and (b) a question must end with a question mark. The full set of syntax rules for a language is called a ***grammar***. Note that English has sentences but programming languages have ***statements***.

People are amazing. They have an ability to read very badly constructed sentences and (usually) make sense of them. For example, a friend sent me the message "How R U?". I pointed out to my friend that the message is rubbish, it is not ***syntactically correct***. In other words, it does not follow the rules of English. My friend was surprised that I could not understand the message.

People are way better than computers. If you want to send an instruction to a computer it has to be 100% syntactically correct. Even 99% correct is not good enough. The computer will only understand the instruction if it is 100% correct. It is therefore very important, as a programmer, to be completely familiar with constructing correct statements in a program. The only way to get good at this is by practice. Always keep practicing your programming.

Our programming course includes learning the various types of statements that are available in the C++ language. Each time we introduce a new statement we discuss the syntax (the rules) of how to properly construct that type of statement.

Here is the first syntax rule for C++ and you will use this rule in every program:
A statement in C++ must end with a semi-colon

Semantics

Semantics refers to the ***meaning*** of a sentence in the language (or the logic). Note that perfect syntax does not imply good semantics. For example, the following English sentence has perfect syntax (it follows the rules of English) but has no meaning at all:

"Colourless green ideas sleep furiously" (*Noam Chomsky, 1957*)

Each time we learn a new ***statement*** in C++, we will learn the syntax for the statement (how to write it) and the semantics for that statement (what happens when the computer executes the statement).

The Editor and Compiler for this course

Editors are used to enter program instructions. Compilers are used to translate instructions to machine code. There are a large number of free editors and compilers available for C++ and if you are already using a particular editor/compiler, feel free to continue using it.

We will provide a free, downloadable editor and compiler for you to use.

Extra reading (don't worry – it's not going to be in the exam!)

Find out more about algorithms:
http://computer.howstuffworks.com/question717.htm

Find out more about computer programming:

http://en.wikipedia.org/wiki/Computer_programming

Find out more about programming languages:
https://en.wikipedia.org/wiki/Programming_language

Some information about C++:
https://www.learncpp.com/cpp-tutorial/introduction-to-cplusplus/

Extra online practice (only if you want to, this is not part of our course)

This is a good website to help you to learn C++ programming:
https://www.learn-cpp.org/

We cover most of what is in "the Basics". Warning – this is not our course, it may cover things slightly differently from our course, it is only to get some extra practice.

Notes for C++

Section 1 : Programming basics

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello world!";
}
```

The program consists of two parts:

Part A – the framework. The framework sits around our instructions and looks like this:

```
#include <iostream>
using namespace std;

int main() {
  ...
}
```

The framework provides the structure for your computer program. Each component of the framework means something and we will discuss the components as we move through the course. Right now, we want to focus on our own instructions so, for now, we will use the framework in every program we create and we will discuss the details later.

Part B – the instructions. Inside the framework, we place our instructions. This program has only one instruction which is:

```
  cout << "Hello world!";
```

The instructions are placed inside the **curly brackets** (braces). Instructions in a program are called **statements**. A program can have many statements but this program has only one statement which is a cout statement. The statements are **indented** – leave some spaces (or press tab) at the left-hand margin. Much of our course consists of learning new statements.

The cout statement

The cout statement is used in a C++ program to display text on the screen. All text that is between the quotation marks is displayed on the screen. Try changing the text and see what happens.

```
#include <iostream>
using namespace std;

int main() {
  cout << "My name is ";
  cout << "Bilbo";
}
```

This program has two cout statements and it will display the following text on the screen:

```
My name is Bilbo
```

Each cout statement continues from where the last one finished. Note that there is a space between "is" and "Bilbo" because I carefully left a space after "is" in the first cout statement. If that

space was not there, then the final display would be:

```
My name isBilbo
```

The computer always displays ***exactly*** what is between the quotation marks (including spaces).

Displaying several lines (using endl or \n)

The above program showed how two cout statements ended up displaying only one line of output. But what if we want to display several lines of output? We use the special ***newline*** character. Let us write another program (which must be saved separately from the first two programs):

```
#include <iostream>
using namespace std;

int main() {
  cout << "Bilbo Baggins" << endl;
  cout << "Bag End" << endl;
  cout << "24 Bagshot Row" << endl;
}
```

Note that endl causes the output to move to a new line, so the output will look like this:

```
Bilbo Baggins
Bag End
24 Bagshot Row
```

In place of endl, we can also use \n to move to a new line. This is often more convenient as it is included within the text. In this case, the program could look like this:

```
#include <iostream>
using namespace std;

int main() {
  cout << "Bilbo Baggins\nBag End\n24 Bagshot Row";
}
```

Practice using cout, endl and \n by writing your own programs.

File names and Saving programs

Each program must be saved separately **in its own file**.

Do not attempt to save more than one program together in the same file.

Each file must have a name. Standard file names contain letters and numbers but not any other strange characters. Standard file names do not contain spaces. On some operating systems you can include spaces in the name but the real name of the file will not be the name with the spaces. You can have problems when a file name is not the actual file name so it is a good habit to use standard file names.

Note that every program file name ends with the `.cpp` extension to show that it is a C++ program. If you are using Windows it is important to make the file extensions visible. For instructions on how to do this, google “windows 10 unhide file extensions”.

Save your files on any handy storage device – we usually use a USB flash drive but you could use a hard drive or any other device that is available for saving files. Massey may make storage space available for you but it is always good to have your own backup copy on your own USB.

You must PRACTICE your programming!!

Type in a program! Run the program.

Change the program and run it again. Note how the ***output*** changes.

Extra online practice (only if you want to, this is not part of our course)

This is a good website to help you to learn C++ programming:
https://www.learn-cpp.org/

We cover most of what is in "the Basics". Warning – this is not our course, it may cover things slightly differently from our course, it is only to get some extra practice.

Integer variables

- A variable is a location in the computer memory.
- Each variable has a name.
- Each variable has a type (for example an integer variable)
- Values are loaded into the variables.
- Each variable holds one value at a time but may change (vary) as the program runs.

Here is a program that defines and then uses integer variables:

```
#include <iostream>
using namespace std;

int value1, value2, result;

int main() {
  value1 = 3;
  value2 = 4;
  result = value1 + value2;
  cout << "The result is " << result;
}
```

This program ***defines*** three variables called `value1, value2` and `result`.
These are all integer variables (that is why `int` appears at the start of the definition).
The program loads the value 3 into variable `value1`.
The program loads the value 4 into variable `value2`.
The program adds `value1` to `value2` and loads the answer into variable `result`.
What value will be loaded into the variable `result`?
We display the result on the screen by using cout and the name of the variable.

An ***integer*** is a "whole number" that never has a fraction. It may be negative.
Examples of integers are: `64, -35, 0, 192`

Variable names

Each variable must have a unique name. You can select whatever name you like for your variables. However, it is very useful to give your variables meaningful names. This is not useful to the computer but it is useful to you (the programmer). It helps you to remember what each variable is being used for. In the example above, we used the name `result` for the variable that is loaded with the result of the calculation. Variable names must start with a letter of the alphabet (a to z). You can include digits (0 to 9) in the variable name but not as the first character in the name. You may not include other characters in the name and there must not be spaces in the name.

Assignment Statement

The assignment statement is used to load a number into a variable. For example: `value1 = 3;`
We talk about this as "`value1` is assigned the value of 3".
A variable can be loaded with a number, for example: `value2 = 4;`
A variable can be loaded with another variable, for example: `value2 = value1;`
A variable can be loaded with the result of a calculation, for example: `result = value1 + value2;`

Sequence

The ***order*** in which statements appear is important. Here is an example of the ***wrong order***:

```
result = value1 + value2;
value2 = 4;
value1 = 3;
```

This program will not calculate the correct result because `value1` and `value2` do not have the correct values when the calculation is performed. Try it out and see what happens.

Integer Arithmetic (arithmetic operators)

`a + b`	the add (or plus) operation
`a - b`	the subtract (or minus) operation
`a * b`	the multiply (or times) operation
`a / b`	the divide operation
`a % b`	the remainder (or modulus) operation

Integer Division

When integers are divided, the result is always an integer – there is no fraction.
For example:

The result of `6 / 3` is `2`
The result of `9 / 5` is `1`
The result of `2 / 3` is `0`

The Remainder Operator

The remainder operator returns what is left over after groups have been formed. For example:

The result of `9 % 4` is 1

In other words, we have 9 things and we remove groups of 4 (there are two groups) and then there is one thing left over (so the answer is 1). Other examples:

The result of `3 % 3` is `0`
The result of `9 % 5` is 4
The result of `2 % 3` is 2

BEDMAS

The usual priority rules are followed and brackets are calculated first. Some examples:

The result of `2 + 3 * 4` is `14`

But the result of `(2 + 3) * 4` is `20`

These operations are useful

These operations are useful in many calculations, and we will make use of them as we progress through the paper. You should practise using them with this small program:

```
#include <iostream>
using namespace std;

int x, y, result;

int main() {
  x = 23;
  y = 45;
  result = x % y;
  cout << "The result is " << result;
}
```

Change the numbers and the operation. Try and predict the result before running the program.

A Computer Program that does a useful calculation

Write a program to convert time in seconds to time in hours, minutes and seconds.

There are 60 seconds in a minute and there are 60 minutes in an hour.

For example: 65 seconds is 0 hours, 1 minutes, 5 seconds.

```
#include <iostream>
using namespace std;

int t, hours, minutes, seconds, allminutes;

int main() {
  t = 4527;
  allminutes = t / 60;
  hours = allminutes / 60;
  minutes = allminutes % 60;
```

```
  seconds = t % 60;
  cout << t << " seconds is " << hours << " hours ";
  cout << minutes << " minutes " << seconds << " seconds.";
}
```

Type this program in and run it. Understand what the program is doing.
Test the program with different times! Is it working correctly? How do you know?

A note on variables: I decided to use 5 variables – you can see them defined at the top of the program. You could use less than 5 variables (can you see how?) or you could use more than 5 variables. There are always different ways to write a program. Always give your variables meaningful names as it helps in understanding the program, especially it is a large program with many variables.

An important note on Testing

In order to test a program, you need to **know what the results will be before you run the program**.

For example, 4527 seconds is 1 hours, 15 minutes, 27 seconds

For example, 127 seconds is 0 hours, 2 minutes, 7 seconds

You need to know this before you run the program. When you run the program, you can then check if it is producing the correct results. Set up several ***test cases***.

Do *not* use the computer to provide you with unknown results!! There is no way to check if the calculations are being carried out correctly.

Always test with simple data!!

Input (also known as "reading" data)

We want to be able to enter values into a program.

```
#include <iostream>
using namespace std;

int value, result;

int main() {
  cin >> value;
  result = value + 3;
  cout << "The result is " << result;
}
```

Interesting things to notice in this program:

- the program will ***wait*** until a number is typed and enter has been pressed.
- while the program is waiting – ***nothing happens***. It is important to use a ***prompt*** (see below).
- the number that is typed is then loaded into the variable in the cin statement.

Use **prompts** that tell the user what to do as in this example:

```
#include <iostream>
```

```
using namespace std;

int length, width, area;

int main() {
  cout << "This program calculates the area of a rectangle\n";
  cout << "Type the width of the rectangle and press enter ";
  cin >> width;
  cout << "Type the length of the rectangle and press enter ";
  cin >> length;
  area = length * width;
  cout << "The area of the rectangle is " << area;
}
```

A **prompt** is a cout statement, or several cout statements, that tell the user what data should be entered. This is important to prevent a blank screen while the computer waits for input.

We can read more than one number on a line, for example:

```
#include <iostream>
using namespace std;

int a, b, c;

int main() {
  cout << "Type in three numbers: ";
  cin >> a >> b >> c;
  cout << "The numbers are " << a << b << c;
}
```

There are several ways to enter the three numbers. Some possibilities are:

```
31 2 -51        (there are spaces between the numbers and Enter at the end)
31   2    -51   (there are tabs between the numbers and Enter at the end)
31
2
-51             (each number is followed by Enter)
```

(or any combination of the above)

Whitespace

The term whitespace is used for spaces of any kind in user input – the space, tab or Enter keys (in any combination). The system knows how many values it must read (due to the syntax of the cin statement) so it ignores the whitespace and waits until the correct number of values has been entered.

Exceptional characters inside the double quotes of a cout statement

The following characters are exceptions to the general rule that everything between the double quotes in a cout statement is displayed on the screen:

`\n` – the newline character (can be used instead of endl)

`\t` – the tab character

If we **want** to display " or \, we use an extra \ in front of " or \.

Thus in a cout statement:

\\ causes one backslash to appear on the screen

\" causes the quote symbol to appear on the screen

What would the following cout statement display on the screen?

```
cout << "I said, \"Hi Frodo,\nhow are you?\"";
```

Note:

Exceptional characters are also known as escape sequences. There are more escape sequences available and you can find out more about them here:
https://ciphertrick.com/c-beginners-introduction-to-escape-sequences/

Comments

A comment is a note to yourself or other programmers. The computer completely ignores the comments. We find it useful to leave notes in programs to explain what the program is doing or why we are performing a particular calculation.

There are two types of comments:

```
/* this comment has a beginning and an end */
// this type of comment continues to the end of the line
```

Here is an example of comments (a mix of both types) used to explain a program:

```
#include <iostream>
using namespace std;

/* This program converts a time in seconds to a time in hours,
   minutes and seconds  */

int t, hours, minutes, seconds, allminutes;

int main() {
  cout << "Enter a time in seconds ";
  cin >> t;
  allminutes = t / 60;         // convert the seconds to minutes
  hours = allminutes / 60;     // calculate the hours
  minutes = allminutes % 60; // minutes is what remains
  seconds = t % 60;           // final seconds after removing minutes
  cout << t << " seconds is " << hours << " hours ";
  cout << minutes << " minutes " << seconds << " seconds.";
}
```

When you submit an assignment, it is a good idea to identify yourself as the author. The best way to do this is to place your ID and your name in a comment at the top of your program. For example:

```
// 2301785 Bilbo Baggins
//
```

```
// This is my assignment to add two numbers together

#include <iostream>
using namespace std;

int value1, value2, result;

int main() {
  value1 = 3;
  value2 = 4;
  result = value1 + value2;
  cout << "The result is " << result;
}
```

Notes:

- the C comment (starts with `/*` and ends with `*/`) can comment out any number of lines
- the C++ comment (`//`) comments out until the end of a line. To comment out several lines with this symbol, you need to repeat it at the start of every line.
- you can use any type of comment in a C++ program

The Programming Process

When creating a new computer program, most experienced programmers will follow a process. This process may vary from one program to the next (and also from one programmer to the next). This is not meant to be a fixed rule of how to create software. Some general guidelines are presented below.

1. State the problem:

What needs to be done? Launch a space rocket, model climate change, etc. Write a general overview of what the program is about.

2. Understand the problem:

What is the data? What calculations must be done? Are there any hidden details? There usually are! It is important to start **generating examples**. Everything we do is based on examples. You need a separate example for each situation. For example, the situation with heavy rainfall, the situation with no rainfall, etc. For a complex problem (e.g. a climate change simulation), understanding the problem may take months and there may still be some missing pieces.

3. Construct a solution to the problem (the solution is often an algorithm):

Create the algorithm. Is it similar to an existing algorithm? No need to reinvent the wheel.
Apply the solution to **the examples** in section 2. Does it work? Get the answers!!
For big programs we may move backwards and forwards between sections 2 and 3.

4. Create the program:

Write your program. You do not need to write it in the same order as it will appear when it is complete. For example, it is difficult to think of every variable when you start writing. Add the variable definitions as you go. What input do you need? Design your display. What output do you need? Once again, **work from your examples**.

5. Test the solution using simple examples:

Run your program. Test it on **your examples**. Make sure you know what the answers will be before you run the program. Then run it and check that the correct answers are produced. Do not use computers to produce answers. You need to know what the answers are before you run the program. Make sure your program works for every possible situation that may arise.

The NZ Cash Example

1. The problem: The price on the item is $8.97 but I must pay cash of $9.00
The price on the item is $8.93 but I must pay cash of $8.90

2. Understand the problem: The amount is rounded to the nearest 10.

3. Construct a solution: We will work in cents, thus $8.97 will be written as 897.

Algorithm
Add 5 to the amount in cents (e.g. 897 + 5 = 902)
Divide the total by 10 using integer division (e.g. 902 / 10 = 90)
Multiply the result by 10 to get the final amount (e.g. 90 * 10 = 900)

[***Note – an algorithm is not a program***]

4. Create the program:

```
#include <iostream>
using namespace std;

int price, cash, temp;

int main() {
  cout << "NOTE: enter all amounts as cents.\n";
  cout << "For example, $8.97 is entered as 897\n";
  cout << "Enter the price on the item: ";
  cin >> price;
  temp = price + 5;
  temp = temp / 10;
  cash = temp * 10;
  cout << "The price is " << price;
  cout << " and you must pay " << cash;
}
```

5. Test the program with examples:

Example 1: the item costs 897. Add 5 to get 902. Integer divide by 10 to get 90. Multiply by 10 to produce the final result of 900. Check that it runs correctly.

Example 2: the item costs 893. Add 5 to get 898. Integer divide by 10 to get 89. Multiply by 10 to produce the final result of 890. Check that it runs correctly.

Note: it is important to have at least one example for each different situation.

For this program we have three different situations as follows:

Situation 1 – the situation in which the amount rounds up (e.g. $8.97 becomes $9.00)

Situation 2 – the situation in which the amount rounds down (e.g. $8.93 becomes $8.90)

Situation 3 – the situation where the amount ends in 5 (e.g. $8.95). At the retailers discretion?

What does our program do with situation 3? Is this what every retailer wants?

Make sure that you have covered every possible situation!!

There are some important features in this program:

– we often replace a value in a variable by using the value from the variable to calculate a new result and then loading the new result back into the same variable.
For example: `temp = temp / 10;`

– we have made use of a well-known **algorithm** for rounding, which is:
add 5, replace the result with itself divided by 10, multiply the new value by 10

Test the program with different amounts. Make sure that you have tested all three situations.

Situation 3 is "at the discretion of the retailer". Can we write a program that works for all retailers in NZ? Should the program ask "what must we do with 5 cents – round up or round down?"

Special challenge

** This is only for those who want to investigate more advanced programming:

A) We are not used to dealing with amounts in cents. Modify the program so that it displays:

```
The price is $8.97 and you must pay $9.00
```

In other words, display the dollar signs and the dots as would be expected.

B) Modify the program to handle situation 3 for all retailers.

Assignment statement shortcuts

If the same variable appears on both sides of an assignment, we can use a shortcut.

For example:

`a = a + 7;` we can use the shortcut: `a += 7;`
`b = b * 3;` we can use the shortcut: `b *= 3;`
`c = c % 6;` we can use the shortcut: `c %= 6;`
`temp = temp / 10;` we can use the shortcut: `temp /= 10;`
`cents = cents - 50;` we can use the shortcut: `cents -= 50;`

There are two special cases of even shorter shortcuts:

```
a = a + 1;          a += 1;          a++;
```

all produce the same result (i.e. they add 1 to the value of `a`) – this is known as ***increment***.

and

```
z = z - 1;          z +-= 1;         z--;
```

all produce the same result (i.e. they subtract 1 from the value of `z`) – this is known as ***decrement***.

You do not have to use shortcuts but you must know what they mean.

The very short shortcuts (e.g. `num++;` and `value--;`) are often used in loops – when we get on to writing loops later in the course.

Shortcuts are also known as augmented assignments or compound assignments. For more information: https://en.wikipedia.org/wiki/Augmented_assignment

Notes for C++

Section 2 : the if statement

The if statement allows the program to carry out different instructions in different situations. Here is an example program that uses an if statement:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10";
  }
}
```

If we run this program and enter 3, the program will display
`This value is smaller than 10`

But what happens if we run the program and enter 15? Nothing is displayed.

The syntax of the if statement (or how to write the if statement)

- it starts with the reserved word if
- there is a ***condition*** (or a test) inside ordinary brackets (parentheses)
- there are statements inside the ***body*** of the if statement – inside the curly brackets

The semantics of the if statement (or how does it work?)

If the condition is true, then the statements in the body are executed. Otherwise the statements in the body are ignored.

Conditions

Every if statement has a ***condition*** (a test inside the brackets).
The result of a test is always `true` or `false`.
You need to know the conditions that are available in a C++ program. They include:

`(a < 9)`	is `a` less than 9?
`(a > 2)`	is `a` greater than 2?
`(a == 4)`	is `a` equal to 4?
`(a <= 7)`	is `a` less than or equal to 7?
`(a >= 3)`	is `a` greater than or equal to 3?
`(a != 5)`	is `a` not equal to 5?

Several if statements in a program

A program can have several if statements. For example:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10";
  }
  if (value > 10) {
    cout << "This value is greater than 10";
  }
}
```

This program has two couts but only two of them will be executed. Or maybe only one…?
What does it display on the screen?
What happens if we run the program and enter 15?
What happens if we run the program and enter 10?

People don't like programs that sometimes have a blank display. Always try to avoid this. We can solve this problem in this program by having three if statements:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10";
  }
  if (value > 10) {
    cout << "This value is greater than 10";
  }
  if (value == 10) {
    cout << "This value is equal to 10";
  }
}
```

Logical Operators

We can link two (or more) tests together to make one multiple-condition test. This is done with **AND** and **OR**. The symbol for **AND** is && (two ampersands) and the symbol for **OR** is || (two bars).

The result of the final test is found by looking at the ***truth tables*** for **AND** and **OR**.

Test 1	Test 2	Result of AND
True	True	True
True	False	False
False	True	False
False	False	False

Test 1	Test 2	Result of OR
True	True	True
True	False	True
False	True	True
False	False	False

Here is an example program:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if ((value >= 1) && (value <= 6)) {
    cout << "This is a valid dice throw\n";
  }
  if ((value == 27) || (value == 83)) {
    cout << "This value is either 27 or 83\n";
  }
}
```

Things to note about this program:

- each test is inside brackets and the full combined test is also inside brackets
- the symbol for **AND** is && and the symbol for **OR** is `||`
- use **AND** to check for a ***range of values*** (in this case, from 1 to 6)

Test this piece of program out. What happens? Change to other values and test again.

The third logical operator is called **NOT** (the symbol is !). **NOT** reverses the value of a test.
For example: `( ! (value == 9))` is another way of writing `(value != 9)`.

Priority of Operators

In arithmetic we use the BEDMAS rule to remember the ***priority*** of the operators.

For example: `(2 + 3) * 4` produces a different result from `2 + 3 * 4`.

In logic the rule is **NOTANDOR**. Thus NOT is the most important and AND is more important than OR.

For example, adding a set of brackets changes the result of this condition:

```
  if ((m <= 8) || (m > 15) && (m == 3)) {
```

Now include an extra set of brackets like this:

```
  if ( ( (m <= 8) || (m > 15) ) && (m == 3)) {
```

Test these conditions with m = 8 and you get different results.

Just like with BEDMAS, brackets can override NOTANDOR.

It is important to practice! Here is an exercise to practice. Predict what the output will be before writing and testing the program. Note the position of the brackets.

```
  m = 24;
  if ( (m != 32) && (m != 24) ) {
    cout << "Result one\n";
  }
  if ( ! (m != 32) && (m != 24) ) {
    cout << "Result two\n";
  }
  if ( ! ( (m != 32) && (m != 24) ) ) {
    cout << "Result three\n";
  }
```

Change the value of m from 24 to some other value. Keep practicing!

Nested if statements

The body of an if statement can contain any number of legal C++ statements. An if statement is also a legal C++ statement and can thus appear inside the body of an if statement. When we have one if statement inside another one, we say that we have "nested" if statements. You will see examples of nested if statements as we work our way through the course. Here is an example:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "This value is less than 10 ";
    if (value >= 0) {
      cout << "and is positive.\n";
    }
    if (value < 0) {
      cout << "and is negative.\n";
    }
  }
  if (value >= 10) {
    cout << "This value is greater than or equal to 10\n";
  }
}
```

Note: there could be more than one if statement inside an if statement
Could there be an if inside an if that is inside an if?

The if statement with an else

Previously we wrote a program that sometimes displayed nothing on the screen. We corrected it by adding an extra if statement to cover the case with no output. We could have corrected it by using an `else` with the if statement, like this:

```
#include <iostream>
using namespace std;

int value;

int main() {
  cout << "Enter an integer value ";
  cin >> value;
  if (value < 10) {
    cout << "The value is less than 10";
  } else {
    cout << "The value is greater than or equal to 10";
  }
}
```

This program will ***either*** display `The value is less than 10` ***or*** it will display `The value is greater than or equal to 10`. Note that it will ***never display both***.

The syntax of the if statement with else (or how to write an if with else)

- the if part you already know
- the else part has ***no condition*** (no test)
- there are statements inside the ***body*** of the else part – inside the curly brackets

The semantics of the if statement with else (or how does it work?)

- if the test is true then the statements in the ***body of the if*** are executed.
- if the test is false then the statements in the ***body of the else*** are executed.
- one part of an if/else statement will **always** run, unlike ifs alone, which may or may not run
- an if statement does not have to include else, it is optional

The if statement with else if and else – this example has several versions

Using if with else sorts out situations where there are two mutually exclusive situations. What if there are more than two? We use **if with else if** as many times as necessary, with a final else (which is optional – you could have all else ifs).

Some universities calculate the class of pass based on the final result for a course. (Massey university does not do this). Write a program to display the class of pass.

Version 1 of the program:

```
#include <iostream>
using namespace std;

int result;

int main() {
  cout << "Enter the final result ";
  cin >> result;
  if (result >= 75) {
    cout << "First class pass";
  } else if (result >= 67) {
```

```
    cout << "Upper second class pass";
  } else if (result >= 60) {
    cout << "Lower second class pass";
  } else if (result >= 50) {
    cout << "Third class pass";
  } else {
    cout << "Fail";
  }
}
```

The syntax of the if statement with else if (or how to write the if with else if)

- `else if` is used for the different alternatives and each else if part has a condition
- the use of else for the final alternative is optional (could be if with only else ifs)
- where else is used, its format is as already seen - no condition

The semantics of the if statement with else if (or how does it work?)

- in the same way as one part of an if/else statement will **always** run, one (and only one) part of an if with any number of else ifs and else will **always** run

Another interesting thing to notice in this program:

- **the order in this particular problem is important** – it won't work if you change the order.

Any program can be written in a number of different ways. If you don't like using else if then we can rewrite this program in a different way. Note that this is a separate, new program.

Version 2 of the program:

```
#include <iostream>
using namespace std;

int result;

int main() {
  cout << "Enter the final result ";
  cin >> result;
  if (result >= 75) {
    cout << "First class pass";
  }
  if (result >= 67) {
    cout << "Upper second class pass";
  }
  if (result >= 60) {
    cout << "Lower second class pass";
  }
  if (result >= 50) {
    cout << "Third class pass";
  } else {
    cout << "Fail";
  }
}
```

Oops, version 2 of the program doesn't work. How do we know it doesn't work? You cannot simply remove else if and expect the same result. In fact it sometimes works (e.g. if the result is 38) and sometimes does not work (e.g. if the result is 84). What causes the problem?

Let us write another version of the program to fix the problem.

Version 3 of the program:

```
#include <iostream>
using namespace std;

int result;

int main() {
  cout << "Enter the final result ";
  cin >> result;
  if (result >= 75) {
    cout << "First class pass";
  }
  if ((result >= 67) && (result < 75)) {
    cout << "Upper second class pass";
  }
  if ((result >= 60) && (result < 67)) {
    cout << "Lower second class pass";
  }
  if ((result >= 50) && (result < 60)) {
    cout << "Third class pass";
  }
  if (result < 50) {
    cout << "Fail";
  }
}
```

Interesting things to notice in version 3 of the program:

- it does not use `else if`
- the order is not important – why not?
- does it work for every result? What if the result is 278?

Always test your programs

Run the program several times with different data. Ensure that you test the program for all the ***different situations*** that may arise. For example, this program should be tested for results in every class. The worst problem is a program that works in some situations but not in others (for an example of a bad program, see version 2 of the program).

Example: A program to find the biggest (maximum) number

Take any three numbers, for example 19 27 2. Which number is the maximum? You probably got it right but how did you know what to do? What algorithm did you use to calculate the biggest number? What if there were 10,000 number and not just 3 numbers?

```
#include <iostream>
using namespace std;

int a, b, c, big;

int main() {
  cout << "Enter three integer values\n";
  cin >> a >> b >> c;
  big = a;
  if (big < b) {
    big = b;
  }
  if (big < c) {
    big = c;
  }
  cout << "The biggest number is " << big;
}
```

Interesting things to notice in this program:

- we assume that `a` is the biggest number and then we check against the others
- we keep changing the variable called `big`. Remember this useful trick!
- make an example and use the example to help you design the program

Example: A program to calculate the minimum number of coins

Assume that you owe someone an amount that is less than $1. What is the minimum number of coins that you need to pay them? Example: 90 cents = 50cents + 20cents + 20cents so you need 3 coins. Assume that the amount is in cents, it must be less than one dollar and it must be a multiple of 10.

```
#include <iostream>
using namespace std;

/* calculates minimum number of coins needed to pay an amount.
   The amount must be less than $1 and must be a multiple of 10. */

int cents, total;

int main() {
  cout << "Enter an amount in cents ";
  cin >> cents;
  if (cents >= 100) {
    cout << cents << " is greater than one dollar\n";
  } else if (cents % 10 != 0) {
    cout << cents << " is not a multiple of 10\n";
  } else {
    cout << cents << " cents = ";
    total = 0;
    if (cents >= 50) {
       cents = cents - 50;
       cout << "50c + ";
       total += 1;
    }
```

```
    if (cents >= 20) {
       cents = cents - 20;
       cout << "20c + ";
       total += 1;
    }
    if (cents >= 20) {
       cents = cents - 20;
       cout << "20c + ";
       total += 1;
    }
    if (cents >= 10) {
       cents = cents - 10;
       cout << "10c";
       total += 1;
    }
    cout << " You need " << total << " coins to pay.";
  }
}
```

Interesting things to notice in this program:

- note the `else if` notation
- the order is important – why?
- make an example and use the example to help you design the program
- some of the statements in the else are if statements (ifs nested inside else)
- this program does not display the results well – what is the problem? Can you fix it?

Calculating leap year

Write a program to calculate if a year is a leap year or not. There are two rules:

1) A year is a leap year if it can be divided exactly by 4 but ***not*** exactly by 100.
   For example, 1900 was not a leap year.

2) A year that divides exactly by 400 is a leap year. For example, 2000 was a leap year.

For more information on leap years see:
https://en.wikipedia.org/wiki/Leap_year

Here is a program that calculates if a year is a leap year. Technically, there is a minimum year and if you enter a year before that, the output won't make sense, e.g. do not enter year 1. Let's not worry too much about that and enter reasonable years.

```
#include <iostream>
using namespace std;

int year, leapyear;

int main() {
  cout << "Enter a year ";
  cin >> year;
  if (year % 4 == 0) {
    if (year % 100 == 0) {
      if (year % 400 == 0) {
        leapyear = 1;
```

```
      } else {
        leapyear = 0;
      }
    } else {
      leapyear = 1;
    }
  } else {
    leapyear = 0;
  }
  if (leapyear == 1) {
    cout << year << " is a leap year.";
  } else {
    cout << year << " is NOT a leap year.";
  }
}
```

Interesting things to notice in this program:

- the remainder operation is used to check if a year is divided exactly by 4 or 100 or 400
- the leapyear variable is used to keep track of whether a year is a leap year or not
- the ***nested if statements*** (in other words: one if statement inside another one)

Any program can be written in a number of different ways. Some people find all those else's confusing. Let us rewrite the program in a different way. Note that this is a separate, new program.

```
#include <iostream>
using namespace std;

int year, leapyear;

int main() {
  cout << "Enter a year ";
  cin >> year;
  leapyear = 0;
  if (year % 4 == 0) {
    leapyear = 1;
  }
  if (year % 100 == 0) {
    leapyear = 0;
  }
  if (year % 400 == 0) {
    leapyear = 1;
  }
  if (leapyear == 1) {
    cout << year << " is a leap year.";
  } else {
    cout << year << " is NOT a leap year.";
  }
}
```

Interesting things to notice in this program:

- the word else is not used (does this make the program easier to read?)
- we start the program by making an **assumption** – we set leapyear to 0 which means we are assuming that the year is not a leap year. We may change the leapyear variable later on.

- the order of the first three if statements is very important…. Why?

Testing

You must ***test your programs***. Run the program several times with different data. Ensure that you test the program for all the ***different situations*** that may arise. For example, this program should be tested **(a)** for a year that does not divide by 4 (e.g. 1999), **(b)** for a year that does divide by 4 (e.g. 2012), **(c)** for a year that divides by 100 (e.g. 1900) and finally **(d)** for a year that divides by 400 (e.g. 2000).

Notes for C++

Section 3 : Real numbers, Booleans, libraries, field widths and debugging

Real numbers always include a decimal fraction. Real numbers are used for measuring (e.g. weights, temperatures, etc) and for scientific calculations. Integers never have a decimal fraction and are used for counting. Here is a program that uses real numbers:

```
#include <iostream>
using namespace std;

float x, y, r;

int main() {
  x = 8.5;
  y = 1.33;
  r = x + y;
  cout << "The final answer is " << r;
}
```

Things to note about this program:

- we use `float` to define real variables
- real numbers are always written with a fraction, e.g. `8.5`, `1.33`, `2.0`, etc

Reading Real Numbers

```
#include <iostream>
using namespace std;

float x, y, r;

int main() {
  cout << "Enter the first number ";
  cin >> x;
  cout << "Enter the second number ";
  cin >> y;
  r = x + y;
  cout << "The final answer is " << r;
}
```

Operations with Real Numbers

The operations are similar to those available for integer numbers and include:

`x + y`	the add (or plus) operation
`x - y`	the subtract (or minus) operation
`x * y`	the multiply (or times) operation
`x / y`	the divide operation

Note that the divide operation produces the correct answer (including the fraction) for real numbers. For example: 1.0 / 2.0 produces 0.5 (whereas 1 / 2 produces 0 when using integer division).

At least one of the values involved must be a real number for fractions to appear in the result.

Note that there is ***no remainder operation*** for real numbers.

Mixed mode Arithmetic – some problems

Example 1:

We can use integer and real number variables in the same program. For example:

```
#include <iostream>
using namespace std;

int a;
float m;

int main() {
  a = 3;
  m = a;       /* m will be loaded with 3.0 */
  cout << "m is " << m;
  m = 6.8;
  a = m;       /* a will be loaded with 6 */
  cout << "a is " << a;
}
```

Notes:

- the computer always chops the fraction off a float when converting to an integer. This can be a problem if it isn't what you want to happen.
- but **truncation of a decimal value can be very useful** in a programming problem that requires you **to extract the whole number part out of a real number.**

Example 2:

The following program gives the wrong answer because of the integer division:

```
#include <iostream>
using namespace std;

int num1, num2;
float average;

int main() {
  num1 = 3;
  num2 = 4;
  average = (num1 + num2) / 2;
  cout << "The average is " << average;
}
```

We can fix it by ensuring that we use real values (not integers) in the division.

```
/* Fix, Version 1 - changing 2 to a real number */

#include <iostream>
using namespace std;
```

```
int num1, num2;
float average;

int main() {
  num1 = 3;
  num2 = 4;
  average = (num1 + num2) / 2.0;
  cout << "The average is " << average;
}
```

Note that the calculation is no longer integer division because 2.0 is a real number.

```
/* Fix, Version 2 - including an extra float variable */

#include <iostream>
using namespace std;

int num1, num2;
float temp, average;

int main() {
  num1 = 3;
  num2 = 4;
  temp = num1 + num2;
  average = temp / 2;
  cout << "The average is " << average;
}
```

Note that the calculation is no longer integer division because `temp` is a float variable.

The Boolean Data Type

Variables are always declared with a particular **data type**, for example: `int a, b, c;`

`int` tells the computer that the data type for these variables is **integer**.

Now we introduce the **Boolean data type** which can only be assigned `true` or `false`. For example:

```
#include <iostream>
using namespace std;

bool header;
int num1, num2;

int main() {
  cout << "Enter the first number ";
  cin >> num1;
  cout << "Enter the second number ";
  cin >> num2;
  header = false;
  if (num1 < 10) {
    if (header == false) {
      cout << "List of numbers less than 10\n";
      header = true;
```

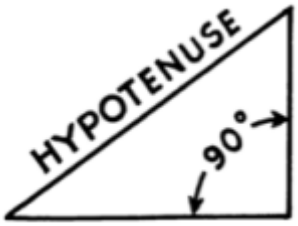
```
    }
    cout << num1 << endl;
  }
  if (num2 < 10) {
    if (header == false) {
      cout << "List of numbers less than 10\n";
      header = true;
    }
    cout << num2 << endl;
  }
  if (header == false) {
    cout << "There are no numbers less than 10\n";
  }
}
```

This program keeps track of whether the header has been displayed or not. If the header has not been displayed then it displays the other message about no numbers less than 10.

Boolean variables are used **to keep track of a situation**. For example, have I already displayed the heading?, is the current year a leap year?, is it raining today? We will discover other programs that use Boolean variables later on in the course. An if statement needs a condition. The condition can be a test or a Boolean variable. Note that you can never read or display Boolean variables.

Libraries and Functions

Calculating the length of the Hypotenuse



$$\text{Hypotenuse} = \sqrt{\text{side1}^2 + \text{side2}^2}$$

A right-angled triangle has side one, side two and the hypotenuse. The hypotenuse is the side opposite the right angle. Pythagoras' theorem (from 500BC) tells us that the square of the hypotenuse is equal to the sum of the squares of the other two sides. Thus:

$$\text{hypotenuse}^2 = \text{side1}^2 + \text{side2}^2$$

Then take the square root of both sides. Now we can write the program to calculate the hypotenuse:

```
#include <iostream>
#include <cmath>    // *IMPORTANT* - an extra #include

using namespace std;

float side1, side2, result, hypotenuse;

int main() {
  cout << "Enter the length of side one ";
  cin >> side1;
  cout << "Enter the length of side two ";
  cin >> side2;
```

```
  result = (side1 * side1) + (side2 * side2);
  hypotenuse = sqrt(result);
  cout << "The hypotenuse is " << hypotenuse << endl;
}
```

Functions and Libraries

Operations have symbols (e.g. `+, -, *, /`). ***Functions*** have a **name** (such as `sqrt`) and they often also have **parameters**. Zero or more parameters appear inside the brackets following the function name. Parameters are also sometimes called ***arguments***. Each function exists within a C++ ***library*** which is a file that contains a collection of functions. Each function can carry out one useful task. We have to `#include` the correct library name in order to use library functions in the program.

Calling a function

We say we **call** a function to use it. The format of a call to a function always involves the function's name followed by parentheses, which may or may not have something in them, depending on whether or not that function has parameters. In some cases the call is part of an assignment statement (see the call to `sqrt` above), but in others it is only the function name and the parentheses. Later we will learn how to write our own functions and the format will be fully explained. For now, we simply learn the format of a call for each library function we use.

The `cmath` library

One of the functions used in the example program above is called `sqrt` and we have to `#include` the `cmath` library to use this function. The `sqrt` function takes one parameter in the brackets. In the above example, the value in the variable called result is the parameter. The `cmath` library contains many other functions that you can use in your programs, such as `sin, cos, fabs, pow` and many more. Here is the full list of available functions in the `cmath` library:
http://www.cplusplus.com/reference/cmath/

The `iostream` library

We have to `#include` the `iostream` library to be able to use input/output functions. `cout` and `cin` are objects in the `iostream` library. More about objects later on.

The `iomanip` library

Provides functions that manipulate the display. For example, see field widths in this section.
http://www.cplusplus.com/reference/iomanip/

The `cstdlib` library

This is a useful library with many interesting functions in it:
http://www.cplusplus.com/reference/cstdlib

Some example functions are:

exit(); *you need to know how to use this function*

`exit()` is used to stop a program. The program will end when the `exit()` function is executed. Put any integer number inside the brackets. Here is an example program:

```
#include <iostream>
#include <cstdlib>

using namespace std;

int value, result;

int main() {
  cout << "Enter a value greater than 10 ";
  cin >> value;
  if (value < 10) {
     cout << "ERROR: the value is not greater than 10";
     exit(13);
  }
  result = value + 4;
  cout << "The result is " << result;
}
```

abs(); *you need to know how to use this function*

`abs()` is used to calculate the absolute value of an integer value. It has one integer parameter. Note that there are other functions (in other libraries) to calculate the absolute value of real numbers (or other data types). In particular `fabs()` in the `cmath` library is used to calculate absolute values for real numbers.

Here is an example program to calculate the age difference between two people:

```
#include <iostream>
#include <cstdlib>

using namespace std;

int age1, age2, difference;

int main() {
  age1 = 25;
  age2 = 27;
  difference = abs(age1 - age2);
  cout << "The age difference is " << difference << " years.";
}
```

rand() and **srand();** *these functions are *not* needed in our course but are interesting*

`rand()` produces a random number. Here is an example program to simulate throwing one dice three times. We also need `srand()` to create a different set of random numbers each time.

```
#include <iostream>
#include <cstdlib>
```

```
using namespace std;

int value, seed;

int main() {
  cout << "Enter the random number seed ";
  cin >> seed;
  srand(seed);
  cout << "Throw a dice three times...\n";
  value = rand() % 6 + 1;
  cout << "The first throw is " << value << endl;
  value = rand() % 6 + 1;
  cout << "The second throw is " << value << endl;
  value = rand() % 6 + 1;
  cout << "The third throw is " << value << endl;
}
```

If you do not have seed (or you enter the same seed each time) the program will give exactly the same sequence of dice rolls. It only becomes random if you enter a different seed each time.

Using namespace

In most programming languages, including C++, every name has to be unique. Thus every function and every variable must have a different name. Sometimes, it can be a problem thinking up unique names. C++ has a system of placing objects within containers (known as namespaces) and this means that names (variables, functions, etc) exist within that namespace. They could then have the same name as another object but this is not a problem as long as the two objects are inside different namespaces.

C++ moved many standard objects and operations into the ***standard namespace*** (called `std` for short). For example, the full name of `cout` is `std::cout` which means that `cout` belongs to the `std` namespace. Similarly, the full name of `cin` is `std::cin`.

We can either use the full names or we can put `using namespace std` at the top of our program which means that `std::` is automatically added to the front of any objects we may be using.

As an experiment, leave out the line `using namespace std` and see what happens.

Using Field Widths with Integer output

```
#include <iostream>
using namespace std;

int value1, value2;

int main() {
  value1 = 37;
  cout << "The value is " << value1 << endl;
  value2 = 1005;
  cout << "The value is " << value2 << endl;
}
```

The display will look like this:

```
The value is 37
The value is 1005
```

Sometimes, we want to line numbers up on the right, like this:

```
The value is   37
The value is 1005
```

Note the **width of the display** (i.e. the number of places on the screen). Each line in this display has a width of 17. Each value in this display has a width of 4. Count the characters on the display to make sure. We can use `cout.width` and `right` (or `left`) to arrange the display as follows:

```
#include <iostream>
using namespace std;

int value1, value2;

int main() {
  value1 = 37;
  cout << "The value is ";
  cout.width(4);              // width of the display will be 4
  cout << right << value1 << endl;
  value2 = 1005;
  cout << "The value is " << value2 << endl;
}
```

Sometimes, we want to show **leading zeros**, like this:

```
The value is 0037
The value is 1005
```

```
#include <iostream>
using namespace std;

int value1, value2;

int main() {
  value1 = 37;
  cout << "The value is ";
  cout.fill('0');          // fill extra places with zeros
  cout.width(4);           // width of the display will be 4
  cout << right << value1 << endl;
  value2 = 1005;
  cout << "The value is " << value2 << endl;
}
```

Note that if a width is too small, the computer ignores it, as in this example:

```
#include <iostream>
using namespace std;

int value;
```

```
int main() {
  value = 8462;
  cout << "The value is ";
  cout.width(2);              // width of the display will be 2
  cout << right << value << endl;
}
```

Using Field Widths with Real number output

The above discussion of width, left, right and fill can also be used with real numbers. In addition, real numbers can make use of `setprecision` to set the number of decimal places to be displayed. Along with `setprecision` we use `fixed` to fix the number of decimal places to be the same for all numbers in the display. Note that we need the `iomanip` library to use `setprecision`.

```
#include <iostream>
#include <iomanip>
using namespace std;

float result;

int main() {
  result = 1.0 / 3.0;   // display this result to 7 decimal places
  cout << "The result is ";
  cout << setprecision(7) << fixed << result << endl;
}
```

Warning – you may not be displaying the correct result

It is possible to display incorrect results. This could be potentially dangerous as an observer may not be aware that incorrect results are being displayed. Here is an example of an incorrect display:

```
#include <iostream>
#include <iomanip>
using namespace std;

float result;

int main() {
  result = 2.0 / 3.0;   // display this result to 1 decimal place
  cout << "The result is ";
  cout << setprecision(1) << fixed << result << endl;
}
```

Example : A program to calculate GST

```
#include <iostream>
#include <iomanip>
using namespace std;

/* this program calculates GST */

float price, gst, total, rate;
```

```
int main() {
  cout << "Enter the price of the item $";
  cin >> price;
  rate = 15.0 / 100.0;     // check that GST is 15.0%
  gst = price * rate;
  /* fix all numbers to display 2 decimal places */
  cout << setprecision(2) << fixed;
  cout << "GST on this item is $" << gst << endl;
  total = price + gst;
  cout << "Total to pay is $" << total << endl;
}
```

Another Example : A program to convert metres to the imperial system

Lengths are measured in New Zealand using the ***metric system***. For example, a basketball court in New Zealand is 28.0 metres long. However, in the USA, distances and lengths are measured using the ***imperial system*** which uses yards, feet and inches. In the USA the length of a basketball court is 31 yards, 1 foot, 0 inches. Are basketball courts actually the same length in the two countries?

Write a program that reads in a length in metres and converts the length to yards, feet and inches. The metres and the inches must contain fractions (real numbers) but the yards and feet do not contain fractions (integer numbers). The output from your program should look like this:

Example 1:

```
Enter a length in metres: 3.497
3.497 metres = 3 yards 2 feet 5.7 inches
```

Example 2:

```
Enter a length in metres: 1.245
1.245 metres = 1 yards 1 feet 1.0 inches
```

(should it be foot or feet..? don't worry about this!)

Note: 1 metre = 39.3701 inches; 12 inches = 1 foot (feet); 3 feet = 1 yard

```
#include <iostream>
#include <iomanip>
using namespace std;

float metres, inches;
int yards, feet;

int main() {
  cout << "This program converts metres into the imperial system\n";
  cout << "Enter a length in metres: ";
  cin >> metres;
  inches = metres * 39.3701;
  feet = inches / 12;          // real division loaded into an int
  yards = feet / 3;
  feet = feet % 3;
  inches = inches - yards * 36 - feet * 12;
```

```
  cout << setprecision(3) << fixed;
  cout << metres << " metres = ";
  cout << yards << " yards " << feet << " feet ";
  cout << setprecision(1) << fixed;
  cout << inches << " inches " << endl;
}
```

Debugging

A problem in a program is called a bug and the process of finding the bugs in a program is called debugging. Programmers spend an enormous amount of time debugging. It is really worthwhile to develop some good habits with your programming style to reduce the amount of time spent debugging.

Software tools to assist with debugging

People have written programs called debuggers which are designed to assist programmers in finding bugs. Generally, debuggers keep track of which instruction the program is currently executing and the current values of all the variables. The use of debuggers in a first-year course is not recommended because they are usually quite difficult to understand and use.

You can read more about debuggers here: (this is not part of our course)
https://en.wikipedia.org/wiki/Debugger

Types of Error

Good programmers know the different types of errors that may occur in a program.

Syntax errors

Errors in the syntax, i.e. the statement is written incorrectly. An example of a syntax error is:

```
  cout << "Hello world
```

The compiler picks up syntax errors, tells you about them, and refuses to run the program. Get used to the syntax errors that are reported by the system (it usually shows you the line number which is useful, and describes what the error is). Make use of these error messages to fix any syntax errors that may occur. Syntax errors are the easiest kind of errors to fix because the compiler tells you about them.

Logic errors

The compiler does not report logic errors and the program runs but produces the wrong results. Logic errors can be very difficult to find because there is nothing wrong with the syntax of the program. For example, you may write a program to add two numbers and the program may run perfectly. But if you were meant to write a program ***multiplies*** two numbers, then you have a program that runs correctly but it is not doing what is required.

The **really difficult logic errors** are when a program works correctly some of the time but not for every different situation that may arise.

In our course we will use these simple definitions of errors:
Syntax error – an error where an error message is reported by the compiler.
Logic error – no error message appears but the program does not do what is required.

There are other types of errors, but we will not worry about them. For example, semantic errors, run-time errors and so on. In fact, many programmers don't agree on the definitions for certain errors!

Make your programs easy to read

Some programmers love to make their programs **easy to write**. Using shortcuts, having short variable names (e.g. a and b) and other tricks make it faster to type in your code. We are not interested in entering code quickly because we know that we don't spend much time on entering code anyway. We spend a huge amount of time going slowly through our programs looking for errors. So, it is a much better idea to make your programs **easy to read**.

Here are some of the ways to make your program easier to read:

- Use meaningful variable names
- Use indentation correctly (especially for nested statements)
- Use appropriate spacing in calculations
- Place a few comments in your program explaining what is being calculated

None of these things are **rules** in the C++ language. But you need to get into good programming **habits** and have good **programming style**. It will help enormously later on when you are trying to debug programs that are much bigger and a lot more complicated.

Display variables as you go

Programmers like to use debuggers to assist with debugging because they keep track of the values in each variable. However, you can always keep track of any variable by including extra cout statements in your program. This is especially important for variables that are not usually displayed as part of the results.

For example, in the following program I use the variable temp. This is an important variable but it is not displayed with the final results. I add some debugging cout statements to the program to display temp.

```
#include <iostream>
using namespace std;

int price, cash, temp;

int main() {
  cout << "NOTE: enter all amounts as cents.\n";
  cout << "For example, $8.97 is entered as 897\n";
  cout << "Enter the price on the item: ";
  cin >> price;
  temp = price + 5;
  cout << "DEBUG: temp after price + 5 is " << temp << endl;
  temp = temp / 10;
  cout << "DEBUG: temp after temp / 10 is " << temp << endl;
  cash = temp * 10;
  cout << "The price is " << price;
  cout << " and you must pay " << cash;
}
```

When the program is complete and after all testing, we can remove those DEBUG cout statements.

Use comments to remove some code in your programs

When there are many lines of code, it is much easier to check only a small part of the program at a time for errors. We can use comments to temporarily remove some instructions. By doing this we can test only the remaining instructions. When everything is working correctly, we can take out the comments and once more include the commented code in the program.

This program has an error in it. Can you see the error? We use comments to remove code temporarily to locate a problem and we include a debugging cout statement.

```
/* A program to find the biggest of three values */
#include <iostream>
using namespace std;

int a, b, c, big;

int main() {
  cout << "Enter three integer values\n";
  cin >> a >> b >> c;
  big = a;
  if (big > b) {
    big = b;
  }
  cout << "DEBUG: The biggest number is " << big;
}
/*if (big < c) {
    big = c;
  }
  cout << "The biggest number is " << big; */
}
```

The debug cout helps to see what is wrong and the commented code will not run. This allows you to focus on the fact that the first `if` statement isn't working properly and then to find the error in that `if`. For long and complicated programs this use of comments is invaluable.

Testing

Testing is extremely important and is the only way to find logic errors. You must always test your programs and you need to test for every possible type of situation. Here are some general notes about testing:

- Set up examples during the programming process (see the lecture on the programming process)
- Set up at least one example for each possible situation
- Make the data in your examples as simple as possible
- Calculate the results yourself before you run the program (know what the result should be)
- Run your program and test every example – check that the results match your examples
- Ask a friend to try out your program

Notes for C++

Section 4 : Loops

The while loop

Here is a program that repeats the same instruction several times:

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello\n";
  cout << "Hello\n";
  cout << "Hello\n";
  cout << "Hello\n";
  cout << "Hello\n";
}
```

We can achieve the same effect in the next program by using a while loop:

```
#include <iostream>
using namespace std;

int counter;

int main() {
  counter = 0;
  while (counter < 5) {
    cout << "Hello\n";
    counter++;
  }
}
```

Notice that there is now a variable (there wasn't one before). This is the ***loop variable*** and counts how many times the loop happens.

The syntax of the while loop (or how to write a while loop)

- it starts with the reserved word while
- there is a ***condition*** (or a test) inside ordinary brackets (parentheses)
- there are statements inside the ***body*** of the loop – inside the curly brackets

The semantics of the while loop (or how does it work?)

Step 1: do the test. If the test is true, do Step 2. If the test is false, ignore the while loop.

Step 2: execute the instructions in the body of the while loop.

Step 3: when you reach the end of the loop (the last closing curly bracket), go back to Step 1.

Walkthrough (also known as a desk check)

Work through the program, doing exactly what the computer would do. Keep track of all changes to any of the variables and also show what will be displayed on the screen. In this example, we keep track of the variable called counter as follows. You can work out what the display would look like.

counter ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ 5

Notice how we ***cross out*** previous values so we can work backwards – if needed to solve an error.

Example : A program to display the numbers from 1 to 10

```
#include <iostream>
using namespace std;

int number;

int main() {
  number = 1;                    // initialise the loop variable
  while (number <= 10) {
    cout << number << endl;
    number = number + 1;         // can we use a shortcut here?
  }
}
```

The ***loop variable*** is the variable that controls the loop. In this program, `number` is the loop variable. It is important to load a value into the loop variable before the loop starts – this is called ***initialising the loop variable***. For homework = modify the program to display numbers from 38 to 54.

Example : A program to keep a running total

We keep a ***running total*** by adjusting the total after each number in the list, as shown in this program to add up the numbers from 3 to 8:

```
#include <iostream>
using namespace std;

int number, total;

int main() {
  total = 0;
  number = 3;
  cout << "Add up the numbers from 3 to 8.\n";
  while (number <= 8) {
    total = total + number;
    number = number + 1;
  }
  cout << "The total is " << total << endl;
}
```

It is important to initialise the total to zero before starting the loop. Running totals must always start at zero. Some people may tell you that variables (including totals) are always set to zero before they are used. This is correct for some variables but not for all variables (more on this later). It is a good habit to always initialise all totals to zero before using them. For homework = modify the program to add up the numbers from 87 to 93. Predict what the total will be before running the program.

Example : A program to enter numbers and keep a running total

We can enter several numbers and keep a running total as shown in this program to add up four numbers:

```
#include <iostream>
using namespace std;

int count, number, total;

int main() {
  total = 0;
  count = 0;
  cout << "Enter four numbers to be added together:\n";
  while (count < 4) {
    cout << "Enter a number ";
    cin >> number;
    total = total + number;
    count++;                        // using a shortcut here
  }
  cout << "The total is " << total << endl;
}
```

We initialise total to zero (as we always do for total variables). We do not need to initialise number as it receives a value that is entered so there is no need to initialise it. We initialise variables that are going to have something added to them (such as total) or are going to be used in a test (such as count). We do not need to initialise variables that are simply going to be loaded with a value later on.

The loop variable in this program is `count`. It keeps track of how many numbers have been entered. We initialise `count` to zero to indicate that, at the start of the program, we have entered zero numbers. For homework = modify this program to read in seven numbers.

Example : A program that uses a trailing value.

The previous program was designed to read in 4 numbers. What if you do not know how many numbers are going to be entered? One way to solve this problem is to use a ***trailing value***. A trailing value is entered after the other numbers to indicate that the data entry has ended. It is important that the trailing value **is not a valid data item**. For example, if we had to enter people's ages, then a good trailing value would be -1 because -1 is not a valid age. -1 is a good trailing value for all sorts of data including student IDs, phone numbers, bank account numbers, etc because it is usually invalid data. However there are some situations where -1 is valid data and (in those situations) you would not be able to use -1 as a trailing value. Here is the program to enter any amount of numbers – make sure you enter -1 after the numbers because we have selected -1 to be the trailing value:

<u>Version 1 of the program that uses a trailing value (this version has some problems)</u>

```
#include <iostream>
using namespace std;

/* a program to read in positive numbers and add them together */
```

```
int number, total;

int main() {
  total = 0;
  number = 999;
  while (number >= 0) {
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
    total = total + number;
  }
  cout << "The total is " << total << endl;
}
```

We notice two things about this program:

- number is initialised to 999. This is because number is used in a test ***before*** it has been entered. Never use a variable in a test before it has been loaded with a value because it is not possible to predict the result of the test. In order to force the test to be true, we stick any value into number. I selected 999 but you could use any positive integer value.
- the program produces incorrect results (did you test it)? This is because the -1 is added to the total. This is exactly what **NOT** to do with a trailing value – it must **NOT** be treated as data.

Let us write a new version of the program to work in a better way:

Version 2 of the program that uses a trailing value (this version works correctly)

```
#include <iostream>
using namespace std;

/* a program to read in numbers and add them together */

int number, total;

int main() {
  total = 0;
  cout << "Enter a number (-1 to stop) ";
  cin >> number;
  while (number >= 0) {
    total = total + number;
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
  }
  cout << "The total is " << total << endl;
}
```

Interesting things to notice in this program:

- if we enter the first value **before** the loop, we don't need 999; we deal only with actual values
- we now need two places where we enter data – once before the loop and again inside the loop
- we have to change the order of the instructions in the loop – why?
- initialising can be done by reading in a value as well as by assigning a value

Another example of a program that uses a loop: The Fibonacci sequence

The Fibonacci sequence appears in nature and is: 0 1 1 2 3 5 8 13 21 …

```
#include <iostream>
using namespace std;

int a, b, c;

int main() {
  a = 0;
  b = 1;
  c = 0;
  cout << "0 1";
  while (c < 200) {   // decide where to stop
    c = a + b;
    cout << " " << c;
    a = b;
    b = c;
  }
}
```

For more interesting information on the Fibonacci sequence:
https://en.wikipedia.org/wiki/Fibonacci_number

Example : Making a program run many times

Loops can be useful to make the task of a program happen repeatedly until the user wishes to stop. For example, a user running a program that reads in a year and prints out whether or not it is a leap year, might want to check several years. We can write a loop that stops when a user enters a specified ending value (a trailing value).

```
#include <iostream>
using namespace std;

int year;
bool leapyear;

int main() {
  cout << "Enter a year (-1 to stop) ";
  cin >> year;
  while (year >= 0) {
    leapyear = false;
    if ((year % 4 == 0) && ((year % 100 != 0) || (year % 400 == 0))) {
      leapyear = true;
    }
    if (leapyear) {
      cout << year << " is a leap year.\n";
    } else {
      cout << year << " is NOT a leap year.\n";
    }
    cout << "Enter a year (-1 to stop) ";
    cin >> year;
  }
}
```

Interesting things to notice in this program:

- we use only one if-statement to check for a leap year
- we have included a loop in the original program (and changed the order of some instructions) in order to repeat the program many times
- decide how to stop the program – in this case we stop if the year is negative

Data validation

In a course, students write two tests. Read in the two test marks for a student and check that each test mark is from 0 to 50. Then add the test marks to calculate the final result.

Version 1 : keep entering the data until it is correct

```
#include <iostream>
using namespace std;

/* read in two test marks (each out of 50) and check for valid data */
/* add the test marks to obtain the final result */

int mark1, mark2, result;

int main() {
  cout << "Enter the mark for test 1 (0 to 50): ";
  cin >> mark1;
  while ((mark1 < 0) || (mark1 > 50)) {
    cout << "ERROR: mark 1 must be from 0 to 50\n";
    cout << "Enter the mark for test 1 (0 to 50): ";
    cin >> mark1;
  }

  cout << "Enter the mark for test 2 (0 to 50): ";
  cin >> mark2;
  while ((mark2 < 0) || (mark2 > 50)) {
    cout << "ERROR: mark 2 must be from 0 to 50\n";
    cout << "Enter the mark for test 2 (0 to 50): ";
    cin >> mark2;
  }
  result = mark1 + mark2;
  cout << "The final result is " << result;
}
```

Interesting things to notice in this program:

- we want to validate (check) that the correct data has been entered
- a loop is included for each data item that will be entered
- if the data is correct, the user will not notice anything (the loops will not run)
- an error message is printed if the data is not in the correct range and the user must enter the data again – this continues until the user enters valid data
- these loops ensure that the calculation only happens on valid data
- we use OR (`||`) to check for values that are outside a valid range

Version 2 : if data is not correct then stop the program

```
#include <iostream>
```

```
#include <cstdlib>          // we use exit from cstdlib
using namespace std;

int mark1, mark2, result;

int main() {
  cout << "Enter the mark for test 1 (0 to 50): ";
  cin >> mark1;
  if ((mark1 < 0) || (mark1 > 50)) {
    cout << "ERROR: mark 1 must be from 0 to 50\n";
    cout << "Program will now close";
    exit(10);
  }
  cout << "Enter the mark for test 2 (0 to 50): ";
  cin >> mark2;
  if ((mark2 < 0) || (mark2 > 50)) {
    cout << "ERROR: mark 2 must be from 0 to 50\n";
    cout << "Program will now close";
    exit(20);
  }
  result = mark1 + mark2;
  cout << "The final result is " << result;
}
```

Interesting things to notice in this program:

- `exit` immediately stops the program
- it doesn't matter what number you put inside the brackets for `exit`
- there is no opportunity to enter the data again
- the user is forced to rerun the program if they enter invalid data
- stopping the program for invalid input means the calculation only happens on valid data

Version 3 : use if's and else's to avoid calculating with invalid data

```
#include <iostream>
using namespace std;

int mark1, mark2, result;

int main() {
  cout << "Enter the mark for test 1 (0 to 50): ";
  cin >> mark1;
  if ((mark1 < 0) || (mark1 > 50)) {
    cout << "ERROR: mark 1 must be from 0 to 50\n";
  } else {
    cout << "Enter the mark for test 2 (0 to 50): ";
    cin >> mark2;
    if ((mark2 < 0) || (mark2 > 50)) {
    cout << "ERROR: mark 2 must be from 0 to 50\n";
    } else {
      result = mark1 + mark2;
      cout << "The final result is " << result;
    }
  }
```

```
}
```

Which data validation technique should you use?

Allow the data to be entered again (version 1) if the data is simple, e.g. test marks. It is really annoying if the whole program stops just because you typed one mark incorrectly.

Stop the program if the data is really complicated (version 2). For example, if the data was a long list of specifications for an engineering project. If complicated data is not entered correctly it may be better to stop the program.

Avoid version 3 as the if's and else's are very hard to read and it is easy to make a mistake in your program.

The for loop

A loop very often contains:

- *initialising* of the loop variable (the variable that 'controls' the loop)
- a *condition* or *test* that involves the loop variable
- a *change* to the loop variable (e.g. m = m + 3;). For example:

```
#include <iostream>
using namespace std;

int counter;

int main() {
  counter = 0;            /* initialising the loop variable */
  while (counter < 5) {   /* the condition */
    cout << "Hello\n";
    counter++;            /* a change to the loop variable */
  }
}
```

In a **for loop** the three parts of the loop (listed above) are collected together inside the brackets. For the example above, we can change the **while loop** to a **for loop** as follows:

```
#include <iostream>
using namespace std;

int counter;

int main() {
  for (counter = 0; counter < 5; counter++) {
    cout << "Hello\n";
  }
}
```

Note how the initialising, the condition and the change now all appear inside the brackets.

Example: A program to find the average (or mean) of 4 numbers

```
#include <iostream>
using namespace std;

int counter;
float number, total, average;

int main() {
  total = 0.0;
  for (counter = 0; counter < 4; counter++) {
    cout << "Enter a number ";
    cin >> number;
    total = total + number;
  }
  average = total / 4.0;
  cout << "The average is " << average;
}
```

Interesting things to notice in this program:

- there is only one variable called `number` but there are many numbers
- note the semi-colons between the three parts of the for loop (not commas)

Example: A program to calculate final results in an exam

```
#include <iostream>
using namespace std;

// there are 8 questions in an exam
// if you get zero for 4 or more questions - you fail
// otherwise you pass if your total mark is >= 50

int count, zeromark;
float mark, total;

int main() {
  total = 0.0;
  zeromark = 0;
  for (count = 0; count < 8; count++) {
    cout << "Enter mark for Question " << count+1 << ": ";
    cin >> mark;
    total = total + mark;
    if (mark == 0.0) {
      zeromark++;
    }
  }
  if (zeromark >= 4) {
    cout << "FAIL - you got zero for " << zeromark << " questions";
  } else {
    if (total < 50.0) {
      cout << "FAIL - you got a total of " << total;
    } else {
      cout << "PASS - you got a total of " << total;
    }
```

```
  }
}
```

Note how count starts at 0 and changes by 1 until it gets up to 8.

Does this program work correctly? Do a ***walkthrough*** to find out.

Incrementing and decrementing

Adding 1 to a variable is called **incrementing**. It can be written as `k = k + 1` or `k += 1` or `k++`

Subtracting 1 from a variable is **decrementing**. It can be written as `k = k - 1` or `k -= 1` or `k--`

Remember that the **change** in a for loop does not always have to be increasing or decreasing by 1.

For example: `k = amount` `k = k * 3` `k = j % 2` `k = k - 5`

When to use a while loop and when to use a for loop?

Use a for loop when you know exactly how many times the loop will happen. Use a while loop in any other situation. This is not a definite rule but is a useful guideline to remember.

Example : A program to find the biggest number (the maximum)

```
#include <iostream>
using namespace std;

int biggest, number;

int main() {
  cout << "Enter the first number (-1 to stop) ";
  cin >> number;
  biggest = number;
  while (number > 0) {
    if (biggest < number) {
      biggest = number;
    }
    /* put an extra cout statement here */
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
  }
  cout << "The biggest number is " << biggest;
}
```

Interesting things to notice in this program:

- we use a while loop and not a for loop – why not?
- the loop variable `number` is initialised by reading it before the loop
- the variable `biggest` is initialised to the first number – why do we do this?
- can you modify this program to find the smallest number (minimum)?

Extra cout statements for debugging

The use of temporary, debugging `cout` statements is always useful. In this program, you could display the values of `biggest` and `number` inside the loop and follow the progress of the program.

Replace the comment `/* put an extra cout statement here */`
with: `cout << "Number is << number << " and max is " << biggest;`

IMPORTANT – the concept of the digits of an integer

An integer number consists of one or more digits. For example, the number 485 consists of 3 digits which are 4, 8, and 5. To 'break' a number into the individual digits, we need a loop, a remainder calculation and an integer division. For example, let us look at reversing a number:

A program to reverse an integer

For example: if the number is 4072 then the reverse of the number is 2704

```
#include <iostream>
using namespace std;

int number, reverse, digit, temp;

int main() {
  cout << "Enter an integer ";
  cin >> number;
  temp = number;
  reverse = 0;
  while (temp > 0) {
    digit = temp % 10;
    reverse = reverse * 10 + digit;
    temp = temp / 10;
  }
  cout << "Number is " << number;
  cout << " and the reverse is " << reverse;
}
```

Does this program work correctly? Do a ***walkthrough*** to find out.

Here is an example of a walkthrough for this program. Make a list of the variables and then work through the program, one instruction at a time. When a variable changes value, cross it out and write the new value next to the variable name.

`number = 3851`

`reverse =` ~~`0`~~ ~~`1`~~ ~~`15`~~ ~~`158`~~ `1583`

`digit =` ~~`1`~~ ~~`5`~~ ~~`8`~~ `3`

`temp =` ~~`3851`~~ ~~`385`~~ ~~`38`~~ ~~`3`~~ `0`

Usually you would also keep track of what is displayed on the screen at each point. However, in this program there is only one `cout` statement at the end of the program. For this particular walkthrough, the following would be displayed:

```
Enter an integer 3851
Number is 3851 and the reverse is 1583
```

Infinite loops

Make sure that a loop will actually end. A loop that never ends is called an infinite loop.
For example, this loop never ends – why not? What will happen when you run it?

```
#include <iostream>
using namespace std;

int count;

int main() {
  count = 0;
  while (count < 5) {
    cout << "Hello";
  }
}
```

A different approach to loops (while true loops)

It is possible to set up an infinite loop and then use the **break statement** to stop the loop.
For example:

```
#include <iostream>
using namespace std;

int number, total;

/* a program to read in numbers and add them together */

int main() {
  total = 0;
  while (true) {
    cout << "Enter a number (-1 to stop) ";
    cin >> number;
    if (number < 0) {
      break;
    }
    total = total + number;
  }
  cout << "The total is " << total << endl;
}
```

Interesting things to notice in this program:

- this is an infinite loop because the test is always true!
- break stops a loop (moves the processor on to the code after the loop)
- usually break will be inside an if statement
- no need for more than one `cin`, because no loop variable to be initialised before the loop
- the use of the "while true" loop makes the program simpler and easier to read

This program uses the opposite test to the way we previously wrote this program.

The previous program used the test: `while (number >= 0) {`

This program uses the opposite test: `if (number < 0) {`

This is because the standard while loop is asking the test question “must the loop run again?” whereas the if statement inside the “while true” loop is asking the opposite test question which is “must the loop stop?”

Notes for C++

Section 5 : Arrays

There are many programs in which we need to work with a large amount of very similar data. For example, if we wanted to keep data on monthly rainfall for one year we would need 12 variables – something like this:

```
float rainjan, rainfeb, rainmar, ……. , rainnov, raindec;
```

The program would get very long if we read values into these 12 variables, and then added them up, and so on. Instead we use an **array – a collection of variables <u>of the same data type</u> (e.g. all ints or all floats etc) under one name.**

Defining an array

When we create a simple variable the computer sets aside a location with that name. For example:

`float a, b;` this means that we have 2 locations like this:

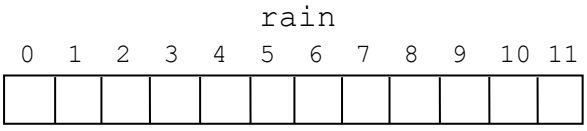


Then the instruction: `a = 4.2;` means that 4.2 is loaded into the box marked `a`.

Defining an array sets aside as many contiguous memory locations as requested, of the data type requested. For example:

`float rain[12];`

this sets up 12 locations like this:



All these locations have the same name `rain`

In the C++ language, we do not declare arrays using a variable to specify the number of boxes required. For example, do not use `float rain[num];`

Index

The name `rain` alone is not enough to work out where to store a value. For example,
`rain = 27.3;`
Where exactly would the value 27.3 be stored as there are 12 variables?

To use one of the variables in an array, we must provide the name **and** the index.
e.g. `rain[5] = 27.3;` would place 27.3 in the box marked 5.

Each variable in an array is called an **element** of the array, e.g. `rain[5]` is an element.

Note that the computer works on the first index being zero. The collection of 12 real numbers above must be declared as `float rain[12];` but the index values are numbered **0 - 11**, not 1 – 12.
The index must be an integer.

Programming with arrays often means repeatedly accessing the same array name at different positions (for assigning, manipulating, printing out etc). This means that loops (especially for loops) are very common in programs that use arrays.

Example: A program to calculate average monthly rainfall

```
#include <iostream>
using namespace std;

int indx;
float total, avg;
float rain[12];

int main() {
  total = 0.0;
  for (indx = 0; indx < 12; indx++) {
    cout << "Enter rainfall for month " << indx + 1 << ": ";
    cin >> rain[indx];
    total = total + rain[indx];
  }
  avg = total / 12.0;
  cout << "Average monthly rainfall is " << avg;
}
```

Interesting things to notice in this program:

- the array is called `rain` and has 12 ***elements*** (variables in the array)
- we read one rainfall figure at a time and load it into the correct location in the array
- we display `indx + 1` so people do not get confused by the computer counting from zero! (people understand January to be month 1 and not month 0)

Notes about arrays

- Each element of the array is a variable with a name and an ***index*** (a number)
- The index can be a variable (eg `rain[indx]`) or a value (eg `rain[5]`) or a calculation

  For example: `rain[(17 + 5) / 2] = 18.4;`

  This leads to the variable (element) `rain[11]` being assigned the value `18.4`
- Beware of going ***out of bounds*** (i.e. using an index that is too big or too small). This will cause the program to produce the wrong results but will not provide an error message.

Example: A program to calculate the maximum and minimum values in an array

```
#include <iostream>
using namespace std;

int indx, maxi, mini;
int numbers[50];

int main() {
  cout << "Enter 50 numbers\n";
  for (indx = 0; indx < 50; indx++) {
    cin >> numbers[indx];
```

```
    /* or you could load a random number into each element of the array     */
  }
  maxi = numbers[0];
  mini = numbers[0];
  for (indx = 0; indx < 50; indx++) {
    if (numbers[indx] > maxi) {
      maxi = numbers[indx];
    }
    if (numbers[indx] < mini) {
      mini = numbers[indx];
    }
  }
  cout << "Maximum is " << maxi << " and minimum is " << mini;
}
```

Interesting things to notice in this program:

- Test this program with less than 50 numbers to save typing! (e.g. maybe 8 numbers)
- `maxi` and `mini` are both initialised to the first number in the array – why is this?
- Can you put an "else" between the maximum check and the minimum check?
- Previously we wrote a program to find maximum (or minimum) without using an array. It is useful to use the array if you need to remember all the individual values for later use.

Counting arrays

We use counting arrays when we want to keep a collection of totals. For example, read in a series of NZ landline phone numbers and count how many numbers there are for each area code.

Landline numbers are 9 digits where the first two digits are the area code, e.g. 09 = Auckland and North, 04 = Wellington, 03 = South Island, etc. The first digit is always zero.

Example 1: the Massey (Auckland) landline is 094140800

Example 2: the Massey (Palmerston North) landline is 063505618

```
#include <iostream>
using namespace std;

int number, indx;
int total[10];           // 10 totals – one for each area

int main() {
  for (indx = 0; indx < 10; indx++) {
    total[indx] = 0;
  }

  cout << "Enter a NZ landline number (-1 to stop) ";
  cin >> number;

  while (number != -1) {
    indx = number / 10000000;
    total[indx]++;
    cout << "Enter a NZ landline number (-1 to stop) ";
    cin >> number;
```

```
  }

  cout << "Total numbers by area code are:\n";
  for (indx = 1; indx < 10; indx++) {
    cout << "Area code ";
    cout.fill('0');         // fill extra places with zeros
    cout.width(2);          // width of the display will be 2
    cout << right << indx;
    cout << " total numbers is " << total[indx] << endl;
  }
}
```

Interesting things to notice in this program:

- the array does not hold the data – it holds the counts associated with the data
- the index is calculated from the data
- some array elements will always be empty, e.g. index = 5 (there is no 05 area code)

Corresponding arrays

A class of students (we don't know exactly how many, but not more than 100) has just written a test (result out of 20). Each student has an ID number of 3 digits. Write a program that reads student IDs into one array and students result into a ***corresponding array***, then searches the ID array for a specific student ID, and prints out that student's result, or the fact that the student was not found.

Note: we will not include data validation (results must be less than 20). You could include this later.

Understanding the problem

We need an array of student ID numbers (an integer array of size 100)

We need an array of student test results of the same size (an integer array of size 100)

These will be used as ***corresponding arrays***, i.e. ID at index `i` corresponds to result at index `i`

For example:

the ID array:

0	1	2	3	4	5
947	281	643	458		

and so on….

the results array:

0	1	2	3	4	5
14	9	16	12		

and so on….

This means that student 947 has a test result of 14 and student 643 has a result of 16, etc.

Construct a solution to the problem

- Read in all the student ID numbers (how many are there?)
- Read in all the test results (how many are there?)
- Repeatedly read in a search ID from the user …
- and run through the ID array checking each element against the search ID…

- displaying either the corresponding result, or the fact that the student was not found

The program

```
#include <iostream>
using namespace std;

int temp, num, select, searchid, i;
int id[100], result[100];

int main() {
// section 1 : enter the student IDs
  i = 0;
  cout << "Enter the student IDs (-1 to stop)\n";
  cin >> temp;
  while (temp != -1) {
    id[i] = temp;
    i++;
    cin >> temp;
  }
  num = i;       // record how many array positions were filled
  cout << "You entered " << num << " numbers\n";

// section 2 : enter the test result for each student
  cout << "Now enter the test result for each student\n";
  for (i = 0; i < num; i++) {
    cout << "Enter the result for " << id[i] << ": ";
    cin >> result[i];
  }

// section 3 : ask for a student ID and search for a student
  cout << "Searching for a student - enter an ID ";
  cin >> searchid;
  select = -1;
  for (i = 0; i < num; i++) {  // run through the ID array
    if (searchid == id[i]) {
      select = i;   // record the id array index of the match
    }
  }
  if (select < 0) {   // if no match was found
    cout << "Student " << searchid << " not found\n";
  } else {
    cout << "Student " << id[select];
    cout << " has result " << result[select];
  }
}
```

Interesting things to notice in this program:

- we use a while loop in section 1. Why?
- we use for loops in the other sections. Why?
- we load the ID values into a temporary variable first, to avoid loading the trailing value into the array. Does this really matter?

- the search was on the id array, but the required data was from the corresponding position in the results array.
- checking an array element by element for a search value, is an algorithm called Linear Search.

When to stop searching

It is important to remember that there are always two different ways to end a search:

(a) you may find what you are looking for (usually known as "success")

(b) you may not find what you are looking for (usually known as "failure")

Note that possibility (b) is ***not*** an error. It is simply the situation in which you search for an item that is not in the array. You must make sure that your program can handle both of these situations.

We do not have to use `temp`

We could write this program without using `temp`. It would look like this:

```
// section 1 : enter the student IDs
  i = 0;
  cout << "Enter the student IDs (-1 to stop)\n";
  cin >> id[i];
  while (id[i] != -1) {
    i++;
    cin >> id[i];
  }
```

This means that we will load the trailing value into the array. However we will never look at the trailing value because it will be stored in element id[num] and we only look at elements where the index is less than num.

Searching algorithms

There are several well-known searching algorithms, i.e. different approaches to searching. We have used the linear search algorithm which runs through the data from the beginning to the end searching for a specified data item. This is not the most efficient searching algorithm (but it is easy to understand and program).

One way of making searching more efficient is to sort the data before you search through it. For example, if you have a long list of names and phone numbers, you do not usually use linear search to find the name you want. You would probably sort the names to make the searching more efficient.

Sorting

People always sorted numbers (and names) long before we used computers. Just to get the feel of how sorting works, let us work through a simple example without using a computer. Take some numbers:

```
48    13    27    62    9
```

Now sort the numbers (in your head) and then write them down in sorted order:

```
9     13    27    48    62
```

How did you sort them? We now need to develop an algorithm for sorting numbers.

An algorithm to sort numbers

Move along the numbers. Compare each number with the following number. If they are in the wrong order, swop them around. For example (**bold** indicates which number we are looking at):

48 13 27 62 9

We look at 48 and 13. They are in the wrong order so we swop them around to get:

13 **48** 27 62 9

Now we move on one position and look at 48 and 27. Again we swop them to get:

13 27 **48** 62 9

We move on one position and look at 48 and 62. There is no need to swop anything.

13 27 48 **62** 9

We move on one position and look at 62 and 9. We swop them to get:

13 27 48 9 62

We have now completed one ***pass*** of the data. A ***pass*** means that we have moved through the numbers from the beginning to the end. In fact we stopped one before the end – why?

However, we have not completed the sorting. Notice that 9 (for example) is still in the wrong position. How many passes will we need to ensure that 9 is placed in the correct position?

How to swop values

During sorting we need to swop two values. We achieve this with the following instructions:

```
int a, b, temp;

a = 3;         /* place an initial value in a */
b = 5;         /* place an initial value in b */
temp = a;      /* 3 lines needed to swop the two values */
a = b;
b = temp;
```

Note that we need an extra variable called `temp`. Why is this?

Note that the extra temporary variable needs to be of the same data type as the data being swopped.

A program to sort an array of integers

```
#include <iostream>
using namespace std;

int num, temp, pass, i;
int data[100];  /* the array can hold a maximum of 100 numbers */

int main() {
```

```
  num = 7;       /* we are working with a total of 7 numbers */
  for (i = 0; i < num; i++) {
    cout << "Enter a number : ";
    cin >> data[i];
  }
  for (pass = 0; pass < num; pass++) {
    for (i = 0; i < num - 1; i++) {
      if (data[i] > data[i+1]) {
        temp = data[i];
        data[i] = data[i+1];
        data[i+1] = temp;
      }
    }
  }
  cout << "The sorted array is ";
  for (i = 0; i < num; i++) {
    cout << data[i] << ", ";
  }
  cout << endl;
}
```

Interesting things to notice in this program:

- `int data[100]` means that the array can ***never hold more than*** 100 numbers
- `num = 7` means that there are currently only 7 numbers in the array
- `for (i = 0; i < num; i++)` will "move along the array". This happens several times.
- the ***outer loop*** (with `pass`) counts the number of passes needed to complete the sorting
- the ***inner loop*** (with `i`) moves along the array and stops one position short of the end
- we swop two ***elements*** of the array. Recall that array elements are variables in their own right.

Extra notes: this sorting algorithm is called Bubble Sort. It can be made considerably more efficient than the version that we have here. There are also many other sorting algorithms. Look for sorting algorithms on Google if you want to see more information.

Deleting and Inserting in sorted arrays

Assume that several student ID numbers have been read into an array and that the ID numbers have been sorted into order. See previous lectures on how to do this. There may be corresponding arrays but we will look only at the ID numbers. Also assume that we know how many ID numbers there are and that this value is stored in `num`.

Deleting from an array to maintain sorted order

A student leaves the class. Delete the ID number `481` and keep all the ID numbers in sorted order. We do this by searching for the index of the ID number to be deleted and then we "shuffle" the ID numbers that are greater than this ID one location to the left.

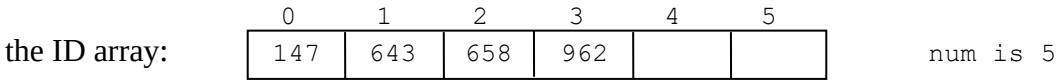
Step 1: search for the position (index) of 481 in the current array

the ID array:

0	1	2	3	4	5
147	481	643	658	962	

```
num is 5
position is 1
```

Step 2: shuffle the ID numbers left starting from position



Step 3: subtract one from num because we have removed one ID number

num is 4

```
#include <iostream>
using namespace std;

int num, removeid, position, i;
int id[100];

int main() {

// section 1: enter five student IDs in the correct order
  cout << "Enter five student IDs in the correct order\n";
  num = 5;
  for (i = 0; i < num; i++) {
    cout << "Enter ID number " << i+1 << ": ";
    cin >> id[i];
  }

// section 2: select one of these IDs to delete
  cout << "Which student ID do you want to delete? ";
  cin >> removeid;

// section 3: search for the position of the specified ID number
  position = -1;
  for (i = 0; i < num; i++) {
    if (removeid == id[i]) {
      position = i;
    }
  }
  if (position == -1) {
    cout << "ID " << removeid << " is not in the array\n";
  } else {

// section 4: shuffle array elements to the left
    for (i = position; i < num - 1; i++) {
      id[i] = id[i+1];
    }
    num--;     // one less ID number in the array
  }

// section 5: display the array
  cout << "\nHere is the full list of ID numbers\n";
  for (i = 0; i < num; i++) {
    cout << id[i] << "  ";
  }
  cout << endl;
}
```

Interesting things to notice in this program:

- there are three situations (cases) to test for – what are they? Check with walkthroughs.
- why is the shuffling loop controlled by `num - 1`, and not `num`?

Inserting into an array to maintain sorted order

A new student joins the class. Insert the ID number `502` into the correct position in the array.
We do this by searching for the correct index for inserting the new ID number and then we "shuffle" the ID numbers that are greater than the new id one location to the right starting from the end/back of the array.

Step 1: search for the correct position to insert 502 into the array

the ID array:

0	1	2	3	4	5
147	481	643	658	962	

num is 5
position is 2

Step 2: shuffle ID numbers right to make a space – start at the back of the array

the ID array:

0	1	2	3	4	5
147	481		643	658	962

num is 5

Step 3: insert the new ID number into the space in the array

the ID array:

0	1	2	3	4	5
147	481	502	643	658	962

num is 5

Step 4: add one to num because we have one more ID number

num is 6

```
#include <iostream>
using namespace std;

int num, newid, position, i;
int id[100];            //NB: the array must have extra space!

int main() {

// section 1: enter five student IDs in the correct order
  cout << "Enter five student IDs in the correct order\n";
  num = 5;
  for (i = 0; i < num; i++) {
    cout << "Enter ID number " << i+1 << ": ";
    cin >> id[i];
  }

// section 2: enter a new ID
  cout << "Now enter a new student ID ";
  cin >> newid;

// section 3: search for the correct position to insert
  i = 0;
  while (newid > id[i]) {
```

```
    i++;
    if (i == num) {
      break;
    }
  }
  position = i;

// section 4: shuffle to the right - start at the end
  for (i = num; i > position; i--) {
    id[i] = id[i-1];
  }

// section 5: insert the new ID number
  id[position] = newid;
  num++;

// section 6: display the array including the new ID number
  cout << "\nHere is the full list of ID numbers\n";
  for (i = 0; i < num; i++) {
    cout << id[i] << "  ";
  }
  cout << endl;
}
```

Interesting things to notice in this program:

- there are three situations (cases) to test for – what are they? Check with walkthroughs.
- note the break in the loop in section 3 – why do we need the break?
- step 2 of the algorithm showed a blank space in the array. Is it actually blank?
- what is the problem with using an exact array size?

Arrays of variable size -- this is NOT part of our course

In certain versions of C++ it is possible to write this:

```
int main() {
  int n;
  cout << "How big is your array? ";
  cin >> n;
  float data[n];
  // code that works with the array
}
```

This gives the impression that an array can be created where the number of elements is not defined until the program runs and the user enters a value for n. This is not the case. What appears to be an array in this program is not actually an array (because arrays always have a fixed size). It is some other data structure that uses pointers and allocates memory (etc). This will be highly inefficient compared to an array. If you want to use a different data structure then be clear about which data structure you are using and be aware of the advantages and disadvantages of the various data structures. Advanced data structures are covered in 159.201.

Notes for C++

Section 6 : Functions

Definition

A function is a named self-contained section of a program that performs a logical task. A function can be thought of as a "mini-program" with variables, statements, etc. Some functions have been created by other programmers (e.g. `sqrt`) and these functions are available to us from the libraries (e.g. `cmath`). This section is about functions that we write for ourselves. These are known as ***user-defined functions.***

Example: A simple function

```
#include <iostream>
using namespace std;

void prompt();                                  /* function prototype */

int x;

int main() {
  prompt();                                     /* function call */
  cin >> x;
  cout << "You entered " << x;
}

void prompt() {                                 /* function definition */
  cout << "Please type in a number\n";
  cout << "The number must be from 0 to 10\n";
  cout << "Press Enter after typing the number\n";
}
```

Interesting things to notice in this program:

- the function prototype and the first line of the function definition are the same
- when `main` ***calls*** the function, `main` stops temporarily, while the statements inside the function definition are executed – in this example there are 3 statements inside the function
- when the function ends (completes all its instructions) then main continues
- we use the word `void` – this will be discussed further on in this section

Program layout

- **function prototype –** void before the name, empty parentheses after the name. The prototypes for all functions appear at the top of the program.
- **function call –** the name and empty parentheses. Appears somewhere in main, could appear several times, could appear inside another function.
- **function definition –** the instructions to carry out the task. Appears after main and each function has its own definition. This is the "self-contained section" of code.

Example: Reading two test results – version 1 (no parameters)

```
#include <iostream>
using namespace std;

void readmark();

int test1, test2, maxmark, temp, result;
/* note - test 1 is out of 30 and test 2 is out of 70 */

int main() {

// section 1: get the mark for test 1
  maxmark = 30;
  readmark();
  test1 = temp;

// section 2: get the mark for test 2
  maxmark = 70;
  readmark();
  test2 = temp;

// section 3: calculate the final result
  result = test1 + test2;
  cout << "Final result is " << result;
}

void readmark() {
  cout << "Enter the mark for the test ";
  cin >> temp;
  while ((temp < 0) || (temp > maxmark)) {
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> temp;
  }
}
```

Interesting things to notice in this program:

- the function is called more than once
- when we want to use the function, we have to remember to set the correct maximum mark before calling the function – we use `maxmark` for this.
- after using the function, we have to remember that the test result is stored in `temp` – we have to copy this to a different variable for each test result.
- is there a better way of passing data and results to and from functions? Yes, there is and we will cover this in the next few examples.

Example: A function with one parameter

```
#include <iostream>
using namespace std;

void prompt(int maxi);                  /* function prototype */
```

```
int a, b;

int main() {
  prompt(33);                          /* function call */
  cin >> a;
  prompt(67);                          /* function call */
  cin >> b;
  cout << "You entered " << a << " and " << b;
}

void prompt(int maxi) {                /* function definition */
  cout << "Please type in a number\n";
  cout << "The number must be from 0 to " << maxi << endl;
  cout << "Press Enter after typing the number\n";
}
```

When this program runs, the display on the screen will look like this:
Note that in this example we assume that the person using the program types the numbers 10 and 55.

```
Please type in a number
The number must be from 0 to 33
Press Enter after typing the number
10
Please type in a number
The number must be from 0 to 67
Press Enter after typing the number
55
You entered 10 and 55
```

Interesting things to notice in this program:

- the function has a ***parameter*** (or ***argument***) called `maxi`
- when the program ***calls*** the function, it ***passes*** a value to `maxi` in the function's area of memory
- the value passed to an `int` or `float` parameter is a **copy** of the value the function is called on
- the data type of the value the function is called on matches the data type in the prototype
- the function uses the parameter as a normal variable (an integer variable in this example)
- parameters can only be accessed by the function they belong to e.g. if you use `max` in main you would get an error
- a function can be called on an actual value, or a variable, or a calculation, for example:
  `prompt(67);` (67 passed to `maxi`)
  `prompt(a);` (10 passed to the function in the example above)
  `prompt((a + 50) / 2);` (30 passed to `max` in the example above)

Example: A function with two parameters

```
#include <iostream>
using namespace std;

void prompt(int mini, int maxi);       /* function prototype */

int a, b;

int main() {
```

```
  prompt(12, 33);                    /* function call */
  cin >> a;
  prompt(85, 99);                    /* function call */
  cin >> b;
  cout << "You entered " << a << " and " << b;
}

void prompt(int mini, int maxi) {      /* function definition */
  cout << "Please type in a number\n";
  cout << "The number must be from " << mini;
  cout << " to " << maxi << endl;
  cout << "Press Enter after typing the number\n";
}
```

Interesting things to notice in this program:

- the function has ***two*** parameters – note the syntax: `int mini, int maxi`
- when the program ***calls*** the function, it ***passes*** two values to the function
- the values are loaded into the parameters **in the order given** (e.g. `mini` gets 12 and not 33)
- the number, order and data types in the function call match those in the prototype
- the function uses the parameters as normal variables
- the parameters are unknown inside main (use of `mini` or `maxi` in main would be an error)

Example: Reading two test results – version 2 (with a parameter)

Look back at the example program to read test results – version 1. We had to use a global variable to send data to the function. Let us write that program again using parameters.

```
#include <iostream>
using namespace std;

void readmark(int maxmark);

int test1, test2, temp, result;

/* note - test 1 is out of 30 and test 2 is out of 70 */

int main() {
  readmark(30);
  test1 = temp;
  readmark(70);
  test2 = temp;
  result = test1 + test2;
  cout << "Final result is " << result;
}

void readmark(int maxmark) {
  cout << "Enter the mark for the test ";
  cin >> temp;
  while ((temp < 0) || (temp > maxmark)) {
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> temp;
  }
```

```
}
```

Interesting things to notice in this program:

- `maxmark` has changed from being a global variable to a parameter
- we no longer have to load `maxmark` before calling the function
- the function is "stand-alone", i.e. not connected to a variable in main (except temp…?)
- the code is more readable and less likely to have errors

Why do we use parameters?

- Parameters are used to send data to the function. For example, the `sqrt` function has to be told which number it must find the square root of.
- Parameters allow the same instructions in the function to be reused for different values. For example, in the program above, the function `readmark` can be used for a test where the maximum mark is 30 and it can also be used for a test where the maximum mark is 70.

Data Diagrams

It may be useful to use simple diagrams to illustrate how data moves between main and the function.

Memory layout for a function with no parameters



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). No data moves directly between main and the function. This is the reason for the word `void` in the prototype and the empty parenthesis after the name. The function's area of memory is only used during the execution of the function (as the instructions of the function are being executed). However, both main and the function can access the ***global variables***, i.e. the variables that are declared at the top of the program.

Memory layout for a function with parameters



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). Copies of the data can be passed directly from main to the function by using parameters. The values to be passed appear in the parenthesis when the function is called. In the function's world, the parameters act as variables along with any local variables. Both main and the function can access the ***global variables*** (i.e. the variables that are declared at the top of the program) but neither can access the either one's ***local variables***.

return and local variables

We have seen the use of parameters **passing** data from a calling function to another function. Now we consider data transferred in the opposite direction – from a function back to the calling function. This is known as **returning** data.

The function prototype must indicate whether or not there is a return, just as it indicates whether or not there are parameters. In the prototype, parameters are indicated in the parentheses and returns are indicated before the name. Note that `void` before the name means no data will be returned.

IMPORTANT: a function may have several parameters, but there is only ever ONE value returned.

Example: A function with parameters (arguments) that returns a value

```
#include <iostream>
using namespace std;

float addthree(float a, int b, int c);

int x;
float y;

int main() {
  x = 3;
  y = addthree(2.0, x, x * 6 % 7);
  cout << "The result is " << y;
}

float addthree(float a, int b, int c) {
  float result;
  result = a + b + c;
  return result;
}
```

Interesting things to notice in this program:

- the function name has `float` in front of it and not `void` because a real number is returned
- put `void` in front of a function that is not going to return any result
- the function ***returns*** a value to main which is loaded into variable `y`
- the call to the function is an assignment statement in order to store the returned value
- `y`, `result` and the start of the prototype are all the same data type (`float` in this example)
- if the prototype shows a return, the word `return` must appear somewhere in the definition.
- a function ends (and the calling function resumes) either when the processor reaches the function's closing brace or, when a return statement runs
- the function has a ***local variable*** that belongs only to the function

Local variables

Local variables live inside a function. They are used for storage inside the function. main (and other functions) do not know about the local variables that belong to other functions. They help to make "stand-alone" functions, i.e. there are no connections between the function and main. This makes functions very portable, i.e. a function can easily be copied into another program or the function can be stored in a library from where it can be used by other programs.

Global variables are declared at the top of the program. All our programs up to now have used global variables. Global variables can be used by main and also by any function in the program.

main is a function. It has a name (main) and it has a return data type (int) and it has parenthesis. main can also have local variables. Any variable that is used only for storage inside main (and not in any function) can be declared as a local variable in main and not as a global variable. This makes programs easier to understand as it is more obvious which variables are used where.

Example: Reading two test results – version 3 (two parameters and return)

Look back at the example program to read test results – version 2. We had to use a global variable to send data to the function. Let us write that program again using parameters.

```
#include <iostream>
using namespace std;

int readmark(int n, int maxmark);

int test1, test2, result;

/* note - test 1 is out of 30 and test 2 is out of 70 */

int main() {
  test1 = readmark(1, 30);
  test2 = readmark(2, 70);
  result = test1 + test2;
  cout << "Final result is " << result;
}

int readmark(int n, int maxmark) {
  int mark;
  cout << "Enter the mark for test " << n << ": ";
  cin >> mark;
  while ((mark < 0) || (mark > maxmark)) {
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> mark;
  }
  return mark;
}
```

This is the third time we have written this program. The first time we had no parameters and no return. The second time we used parameters but no return. This time we use parameters, return and local variables.

This version of the function is now completely **stand-alone**. It has no connection with main. Note that the variable `temp` has disappeared from this version of the program. The function does not make use of any global variables. The function is now portable – ready to be copied into another program (or put in a library).

This version of the program has three global variables which are:

```
int test1, test2, result;
```

We note that the function does not use any of these variables. We could therefore make these variables local variables in main.

How useful is `return`?

1. We can send a result back to main (or the calling function)
2. We can stop the function at any point

It is useful to remember that return does these two things (not just one of them). We can make use of this information in order to make functions simpler and easier to read. Here is an example:

Example (a): The absolute value function (*using return only to send back a result*)

Actually we don't need a function to calculate absolute value because we can use library functions to do this. But it makes an interesting example:

```
int absolute (int x) {
  int result;
  if (x > 0) {
    result = x;
  } else {
    result = -1 * x;
  }
  return result;
}
```

This is only the function definition. Write a `main` program that calls the function. Programmers often look only at functions without looking at the entire program.

Example (b): The absolute value function (*also using return to stop the program*)

Let us write the absolute function again and make it simpler and easier to read:

```
int absolute(int x) {
  if (x > 0) {
    return x;
  }
  return -1 * x;
}
```

Interesting things to notice in this function:

- the word `return` occurs more than once
- there is no local variable and there is no else

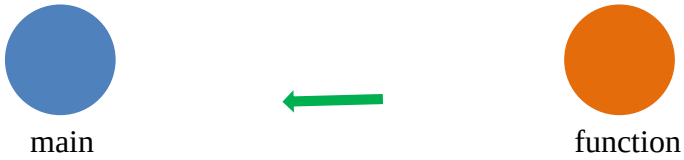
Void functions

A function that does not return a value is called a void function. Instead of a data type, such functions have the word `void` stated before the function name. We have already looked at several examples of void functions. Void functions do not need a return statement. However, you can include a return statement if you want to stop the function early. In this case, the statement would be `return;` as there is no value to return to main.

Data Diagrams

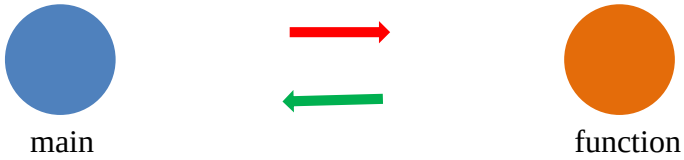
It may be useful to use simple diagrams to illustrate how data moves between main and the function.

Memory layout for a function with a return



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). No data moves directly from main to the function (i.e. no parameters). The function's area of memory is only used during the execution of the function (as the instructions of the function are being executed). Data is returned from the function to main. The data is loaded into a variable in main by using an assignment statement. The data type must be stated before the name of the function. Both main and the function can access the ***global variables*** (i.e. the variables that are declared at the top of the program) but neither can access the either one's ***local variables***.

Memory layout for a function with parameters and return



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). Copies of the data can be passed directly from main to the function by using parameters. The values to be passed appear in the parenthesis when the function is called. In the function's world, the parameters act as variables along with any local variables. The function's area of memory is only used during the execution of the function (as the instructions of the function are being executed). Data is returned from the function to main. The data is loaded into a variable in main by using an assignment statement. The data type must be stated before the name of the function. Both main and the function can access the ***global variables*** (i.e. the variables that are declared at the top of the program) but neither can access the either one's ***local variables***.

Array parameters

We can send an array to a function as a parameter. Array parameters do not have a fixed size. We use the empty square brackets to indicate that the parameter is an array but the size is not given. This means that the function can be used several times with different arrays of different sizes. It is important to include another parameter to inform the function how many elements of the array we are interested in.

Example: A function to calculate the average of numbers in an array

```
#include <iostream>
using namespace std;

float getaverage(float arr[], int count);
```

```
// there are no global variables

int main() {
  float results[100], average;
  int num, i;
  cout << "How many test results do you have? ";
  cin >> num;
  for (i = 0; i < num; i++) {
    cout << "Enter result " << i+1 << ": ";
    cin >> results[i];
  }
  average = getaverage(results, num);
  cout << "The class average is " << average << endl;
}

float getaverage(float arr[], int count) {
  int i;
  float total, result;
  total = 0.0;
  for (i = 0; i < count; i++) {
    total = total + arr[i];
  }
  result = total / count;
  return result;
}
```

Interesting things to notice in this program:

- the function has three local variables – main may not use these variables
- main has four local variables – the function may not use these varables
- the function has a local variable called `i` and main also has a local variable called `i`. It is important not to have a global variable called `i` which could cause confusion.
- the function is **stand-alone** (does not use any global variables). This means that it could be put in a library for other programmers to use.
- there are no global variables at all. This is good practice because it ensures that any functions are stand-alone. However, there are times when it is useful to use global variables.
- the function will work on an array of any size – `count` informs the function how many elements of the array we are actually interested in.
- what happens if I enter more than 100 test results?

Notes for C++

Section 7 : More Functions

Overview of Section 7

Week 7 is a catch-up week and continues with the important topic of Functions which starts in Section 6 of the notes available on Stream under Week 6.

Functions are an important part of programming and you are encouraged to design functions for your own programs and assignments. Assignments may not specifically ask for functions but you need to decide if it would be worthwhile to include functions. If an assignment (or quiz) specifically asks for a function, then make sure you include the function.

There is NOT a rule that says "every program you ever write must have a function in it".

Look under the heading "Why do we use functions?". If you are writing a program and you believe that it would be worth using a function, then go ahead and include a function. In particular, look at the notes about how "Functions enable us to test the main program at an early stage" and "Functions make a program more readable". Many people assume that the main purpose of functions is to avoid duplication of code. While it is useful to avoid duplication, this is not the main reason to use functions.

A reminder about Local and Global variables

Global variables are declared at the top of the program – main and all the functions know about the global variables and can access them.

Local variables (like parameters) exist only inside the function where they are declared – other functions cannot access them.

Why do we use local variables?

- Local variables make the function completely independent of main. The function has its own "world" with data coming in (as parameters) and any extra variables needed are local and completely separate from main.
- This really helps us when we are isolating a logic error inside a function as the function world has no connection to main.
- This also helps to make functions **portable**. If the function is completely separate from main we can confidently copy the function and use it in a new program (or put it in a library for other programmers to use).

Example: Showing the priority of local variables

Program A

```
#include <iostream>
using namespace std;
```

Program B

```
#include <iostream>
using namespace std;
```

```
void ping();

int a, b;

int main() {
  a = 3;
  b = 5;
  ping();
  cout << "a = " << a;
  cout << "  b = " << b;
}

void ping() {
  a = 27;
}
```

```
void pong();

int a, b;

int main() {
  a = 3;
  b = 5;
  pong();
  cout << "a = " << a;
  cout << "  b = " << b;
}

void pong() {
   int a;
   a = 27;
}
```

Answer – Program A displays: `a = 27 b = 5` and Program B displays: `a = 3 b = 5`

Note that Program B has a local variable. Thus the value 27 is loaded into the local variable and does not affect the global variable (which is loaded with 3). In the case of Program A, there is no local variable and so the function loads the value 27 into the global variable.

A program with functions – checking student course enrolment

Most students enrol in a degree that has some core (compulsory) courses. The university needs to keep track of how many core courses each student is taking. Write a program that works for just one student in one particular degree. (Advanced challenge: make this work for 100 students taking one of 3 different degrees, each with different core courses).

This program first establishes how many core courses there are in the degree (in this case there are three) and then loads the course numbers of the core courses into the array called `core`. Then the program loads the courses that the student is taking into the array called `courses`. Finally, the program counts how many core courses the student is doing and displays the result.

```
#include <iostream>
using namespace std;

bool checkcore(int corecrs[], int count, int coursenum);

int main() {
  int num, numcore, i, temp, corecount;
  int core[10], courses[40];

// section 1 : load the core courses
  numcore = 3;
  core[0] = 159101;
  core[1] = 161111;
  core[2] = 247112;

// section 2 : load the courses for one student
  i = 0;
  cout << "Enter a course for this student (-1 to stop) ";
  cin >> temp;
```

```
  while (temp != -1) {
    courses[i] = temp;
    i++;
    cout << "Enter a course for this student (-1 to stop) ";
    cin >> temp;
  }
  num = i;
  cout << "This student is taking " << num << " courses.\n";

// section 3 : count how many core courses
  corecount = 0;
  for (i = 0; i < num; i++) {
    if (checkcore(core, numcore, courses[i])) {
      corecount++;
    }
  }
  cout << corecount << " of the courses are core courses.\n";
}

bool checkcore(int corecrs[], int count, int coursenum) {
  int i;
  for (i = 0; i < count; i++) {
    if (coursenum == corecrs[i]) {
      return true;
    }
  }
  return false;
}
```

Interesting things to notice in this program:

- there are no global variables and all the variables are local.
- in this program it is very important for main and the function to each have their own variable called `i`. This is because `i` in the function gets changed many more times than the `i` in main.
- core courses are sent as an array parameter to the function
- only one element of courses is sent to the function. This is a normal integer parameter (not an array parameter) because the function only looks at one course at a time.
- the function is a **Boolean function** (it returns true or false). Boolean functions are very useful when something has to be checked – in this case, is this course a core course?

A program with functions – what day of the week is the ANZAC Day holiday?

Write a program to calculate what day of the week the ANZAC Day holiday will fall on.

Understand the problem:
What date is ANZAC day? is the holiday always on that date? (no, it is sometimes shifted to the following Monday.) If we know the day of the week for 01 April then ANZAC Day is 24 days later. Invent a code for the days of the week, for example 0 = Sun, 1 = Mon, 2 = Tues, 3 = Weds, 4 = Thurs, 5 = Fri, 6 = Sat (but you could invent a different code).

Create some examples:
Example 1: 01 April in 2019 is on a Monday (code = 1) and the holiday is on a Thursday (code = 4)
Example 2: 01 April in 2020 is on a Weds (code = 3) and the holiday is on a Monday (code = 1)
How many examples do you need? At least one example for each different situation that may arise.

Construct a solution:
The weekday code for ANZAC Day = code for 01 April + 24
But we are working with weeks (groups of 7 days) so ANZAC code = 01 April code + 24 % 7
Watch out for BEDMAS! Remainder is like division so we need brackets: (01 April code + 24) % 7
If ANZAC is a Sunday (code = 0) or Saturday (code = 6) then shift it to a Monday (code = 1)
***** TEST your solution using your examples *****

Write your program:
Use your examples to design the display.
While coding, invent new functions to hide details – keep your focus on main
Use comments when you need to remember why you have done something
Include extra DEBUG cout statements to display variables that are otherwise hidden

```
#include <iostream>
using namespace std;

int aprilstartday();
int anzacday(int day1);
void printday(int dayofweek);

int year, firstday, anzac;

int main() {
  cout << "Enter the year ";
  cin >> year;
  firstday = aprilstartday();
  cout << "April 01 in " << year << " is on a ";
  printday(firstday);
  cout << endl;
  anzac = anzacday(firstday);
  cout << "The ANZAC Day holiday in " << year << " is on a ";
  printday(anzac);
  cout << "\nDEBUG anzac = " << anzac;  // debug - remove later
  cout << endl;
}

int aprilstartday() {
  int temp;
  cout << "0 = Sun  1 = Mon  2 = Tues  3 = Weds";
  cout << "    4 = Thurs  5 = Fri  6 = Sat\n";
  cout << "Using this coding system, enter the day of the week ";
  cout << "for 01 April in " << year << endl;
  cout << "For example, if 01 April is on a Friday then enter 5\n";
  cin >> temp;
  return temp;
}

int anzacday(int day1) {
  int temp;
  temp = (day1 + 24) % 7;  // move from 01 to 25 April
  if ((temp == 0) || (temp == 6)) {
    temp = 1;       // holiday is on Monday if 25th is Sat or Sun
  }
```

```
  return temp;
}

void printday(int dayofweek) {
  if (dayofweek == 0) {
    cout << "Sunday";
  } else if (dayofweek == 1) {
    cout << "Monday";
  } else if (dayofweek == 2) {
    cout << "Tuesday";
  } else if (dayofweek == 3) {
    cout << "Wednesday";
  } else if (dayofweek == 4) {
    cout << "Thursday";
  } else if (dayofweek == 5) {
    cout << "Friday";
  } else if (dayofweek == 6) {
    cout << "Saturday";
  }
}
```

IMPORTANT SECTION: Why do we use functions?

- Functions can avoid duplication of code.
- Functions enable us to focus on one topic at a time.
- Functions can be used to isolate errors and thus assist with debugging.
- Functions enable us to test the main program at an early stage.
- Functions make a program more readable.
- We can save time and effort by using functions from previous programs when creating a new program (or by using functions that are available in a library).

Functions can avoid duplication of code

We can use a function several times in a program. If the function was not available, we would have to type in two sections of code that would be almost exactly the same. The function avoids this. Some people say that duplication of code is the only reason to use functions. You can see this opinion is not correct by looking through the rest of this list.

Functions enable us to focus on one topic at a time and thus assist with program design

For example, in the ANZAC Day program, the topic for the main program is "calculate the day of the week for the holiday". We do not want to get distracted by side issues such as "is the code for Sunday a 0 or a 6?".

When we are writing the main program, we include the line that calls the function:

```
  firstday = aprilstartday();
```

At this stage we do not know the details of how the function will work. We simply know that it will provide us with the weekday code for 01 April in the given year. We can therefore continue designing and writing the main program without distractions. Using this function has simplified designing the

program, breaking the problem down into more manageable parts. When we have finished writing main, we can turn our full attention to what the function needs to do.

Functions can be used to isolate errors and thus assist with debugging

By using functions we can remove complicated sections of code (for example, how to read a test result and check that it is in the correct range) from main. In particular, functions allow us to focus on one small piece of the program and make sure that it is working correctly.

As an example, let us assume that we made a mistake in the `readmark` function. Let us say that we made a typo and entered the first line of the while loop as follows:

```
  while ((temp < 0) || (temp < maxmark)) {
```

Can you see what is wrong with this?

This causes a logic error. The program runs but it does not behave correctly. It will not accept correct test results and it will accept incorrect test results.

When we see this problem we realise that there is something wrong with the function because the task of the function is to read the test results. We can confirm that the problem is in the function by placing an extra `cout` in the function like this.

```
void readmark() {
  cout << "Enter the mark for the test ";
  cin >> temp;
  while ((temp < 0) || (temp < maxmark)) {
/**/ cout << "temp is " << temp << endl;     // an extra cout
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> temp;
  }
```

If we study the output from this extra `cout`, we will work out that the test for the loop is not correct. Then we can find the error and fix it.

All this debugging takes place inside the function. We leave main completely unchanged. In a big program, if we can isolate a logic error inside one function, it is much easier to find what is wrong than if we need to check the whole program.

Functions enable us to test the main program at an early stage

All programs need testing. However, testing becomes complicated if the entire program has to be completed before testing. At that late stage, it is difficult to determine which section of the program (or of a function) is causing the problem. Thus it is better to test the main program at an early stage and fix any errors that are occurring. Then the functions can be developed and tested separately later.

One approach is to write ***function stubs***. An example of a stub for the above program could be:

```
int aprilstartday() {
  return 1;   // Assume that April always starts on a Monday
}
```

A stub is a function that runs and returns a "reasonable" result but we don't have to spend a lot of time designing the function. Also the stub is very simple and so it is unlikely to have any errors in it. In this case the stub will work correctly for 2019 but not for most other years. That's ok, we will test the program only for 2019 (at this stage). The stub means that we can rapidly get to being able to run the

program and any errors that may occur are probably in main. We can then fix any errors that may be present in main before we get back to designing the full detail of the functions.

Functions make a program more readable (and thus easier to debug)

It is important that programs are easy to read. We write a program once but we read it many times (checking for errors) and other programmers may have to read it as well. If a program is easy to read, any errors in it will be quickly discovered. We have several ways to improve the readability of a program including:

- comments – good (but brief) comments explain the purpose of a function, statement etc
- indentation – this clearly shows which instructions are inside a loop, or a function, or… etc
- meaningful names for variables and functions
- the use of functions to break the problem into parts, reducing main to more of an overview of what the program is doing - something shorter and easier to follow

In the ANZAC Day program for example, if the two calls to `printday` were not there, we would have a whole sequence of instructions (twice) and main would be longer and harder to read.

`main` is a function

main is a function. It has a name, it has a return data type (`int`) and it has brackets (with no parameters). And main can have parameters as well but we won't get onto that in our course. Like all other functions, main can have local variables and can also access any global variables.

Notes for C++

Section 8 : Characters and Strings

Characters

Examples of characters: `'A'  'h'  '+'  ' '`. Note that each character is surrounded by single quotes. This is especially important for the space character, otherwise we would not see it! Here is a program that uses a character variable:

```
#include <iostream>
using namespace std;

char k;

int main() {
  k = 'A';
  cout << "The character is " << k << endl;
}
```

ASCII codes

Computers do not store characters – they only store numbers. In order to store characters they have to first be converted to numbers. There is an international set of codes (the ASCII code) that tell us which integer value each character will be converted to. You can find more information here: https://theasciicode.com.ar/

You should know the following ASCII codes:

`'A' is 65.  'a' is 97.  '0' is 48.  ' ' is 32.`

The first three start a sequence, e.g. `'B' is 66` and `'d' is 100` and `'2' is 50`.

Note that we often store digits (or numbers) as characters. For example, phone numbers and ID numbers look like numbers but are not used as numbers. For example, we do not add two phone numbers together.

We can load integer values (or ASCII codes) directly into character variables and we can do arithmetic operations on the integer values stored in character variables. For example:

```
#include <iostream>
using namespace std;

char value;

int main() {
  value = 97;
  value = value + 3;
  cout << "The character is " << value << endl;
}
```

What does this program display on the screen? Note that special characters are also characters.
So we could have this line in a program:

```
  k = '\n';
```

How to input characters

```
#include <iostream>
using namespace std;

char c;
int code;

int main() {
  cout << "Press a key followed by Enter ";
  cin >> c;
  code = c;
  cout << "\nThe character is " << c << endl;
  cout << "\nThe character " << c << " has ASCII code " << code;
}
```

This code works fine most of the time. But it doesn't work for certain characters such as <space>.

NOTE – the next program is NOT part of our course. It is only for those who want to hack deeper. This program uses the function `getchar` (from the `cstdio` library) to read a wider range of characters.

```
#include <iostream>
#include <cstdio>
using namespace std;

char c;
int code;

int main() {
  cout << "\n\nPress a key followed by Enter ";
  c = getchar();
  getchar();       // to flush the enter from the input buffer
  code = c;
  cout << "\nThe character is " << c << endl;
  cout << "\nThe character " << c << " has ASCII code " << code;
}
```

Definitions – digit, letter and punctuation

digit – a digit is one of the characters '0' to '9'. Any number (in the decimal number system) is made up of a series of digits.

letter – a letter is one of the characters 'a' to 'z' and also 'A' to 'Z'. Any word (in English) is made up of a series of letters. Lower case letters are 'a' to 'z' and upper case letters are 'A' to 'Z'. Upper case letters are also known as capital letters (or caps).

punctuation – a set of characters that are used together with letters to form English sentences. They include comma, dash (or hyphen), full stop (or period), exclamation mark, question mark, semicolon, colon, brackets, apostrophe and quotation marks.

Example: How to convert a lower case letter to an upper case letter:

```
#include <iostream>
using namespace std;

char letter;

int main() {
  cout << "Press a key followed by Enter ";
  cin >> letter;
  if ((letter >= 'a') && (letter <= 'z')) {
    letter = letter - 32;
    cout << "The upper case letter is " << letter << endl;
  }
}
```

Note that we convert lower case to upper case by subtracting 32. Study the ASCII codes to see why.

What will happen if you type a character that is not a lower case letter, for example `'A'` or `'+'`?

Modify the program to display an appropriate message in this case.

Change this program to convert upper case letters to lower case.

Example: How to repeat a program with a question:

```
#include <iostream>
using namespace std;

int a, b, result;
char choice;

int main() {
  choice = 'X';    // any character that is not Q
  while ((choice != 'Q') && (choice != 'q')) {
    cout << "Enter two numbers ";
    cin >> a >> b;
    result = a + b;
    cout << "The result is " << result << endl;
    cout << "Enter Q to quit - any other key to continue";
    cin >> choice;
  }
}
```

Interesting things to notice in this program:

- the program adds two numbers – the other instructions are to repeat the program
- we realise that a person may press `'Q'` or `'q'` so we cover both of these situations
- work through and understand the test `((choice != 'Q') && (choice != 'q'))`

Arrays of characters

We can create and use arrays of characters, for example:

```
#include <iostream>
using namespace std;

char data[63];

int main() {
  data[34] = 'h';
  data[57] = 's';
  cout << data[34] << " " << data[57] << endl;
}
```

Strings

We often want to use more than just a single characters. We want to work with names, words and sentences. For this we use a string.

Example: A program that uses a string

```
#include <iostream>
#include <string>
using namespace std;

string msg;

int main() {
  msg = "Good evening to you";
  cout << msg << endl;
}
```

Interesting things to notice in this program:

- you need to #include the `string` library to work with strings
- place double quotes around the text

Example: A program that reads a string (input)

```
#include <iostream>
#include <string>
using namespace std;

string msg;

int main() {
  cout << "Enter your text message: ";
  cin >> msg;
  cout << msg << endl;
}
```

Problem: The input stops when it gets to a space

The program above only works if you enter one word with no spaces. What happens if you enter a message consisting of several words with spaces between them? To be able to read in a line of several words we use the `getline` function from the `string` library.

```
#include <iostream>
#include <string>
using namespace std;

string msg;

int main() {
  cout << "Enter your text message: ";
  getline(cin, msg);
  cout << msg << endl;
}
```

Remember:

`cin` reads one word at a time (stops at a space)
`getline` reads in one line at a time (can include any number of spaces)

Problems with the input buffer

A buffer is a temporary storage area. The input buffer stores any key stroke you make and then passes it on to your program when the program executes an input instruction. Something to tend to forget is that when we enter numeric data, we type a number and then press enter. The enter is stored in the input buffer and passed to the program at the next input instruction. This can cause problems as shown in the next example program.

```
#include <iostream>
#include <string>
using namespace std;

string name;
int age;

int main() {
  cout << "Enter your age: ";
  cin >> age;
  cout << "Enter your name: ";
  getline(cin, name);
  cout << "Your name is " << name << " and your age is " << age;
}
```

Try this program – it does not produce the correct output. This is because the stored enter (after the age) is passed through to the `getline` function and so the name becomes enter. We need to ***flush the buffer*** – clear the enter out of the buffer so that we can enter the name. We can add `getchar` after reading the age – this will read the stored enter and leave the buffer empty and ready to read the name. Remember that you need to #include `cstdio` to be able to use `getchar`

```
#include <iostream>
#include <cstdio>
#include <string>
using namespace std;

string name;
int age;

int main() {
  cout << "Enter your age: ";
  cin >> age;
  getchar();  // this reads the enter and flushes the buffer
  cout << "Enter your name: ";
  getline(cin, name);
  cout << "Your name is " << name << " and your age is " << age;
}
```

A string is an array of characters

A string is an array of characters. Like any array, you can use the index number to access a particular element of the array.

Example: A program that changes a particular character in a string

```
#include <iostream>
#include <string>
using namespace std;

string text;

int main() {
  text = "We are going to sleep";
  cout << text << endl;
  text[17] = 'w';
  cout << text << endl;
}
```

Interesting things to notice in this program:

- just one character is changed
- you need to calculate what the index number is – start from zero and include spaces

Example program: convert characters in a string to a number

Enter a street address, for example, 36 Ocean View Road. It is impossible to know how many spaces there are going to be in the address. Check if the number is less than or greater than 500 (couriers need to do this sort of thing).

Possible solution – read the whole address as a string. Convert from characters in a string to a number. Formula – get a digit and convert from character to number by subtracting 48 (because the ASCII code of zero is 48). Then multiply the number by 10 and add the new digit.

This program looks like this:

```
#include <iostream>
#include <string>
using namespace std;

string input;
int num, i;

int main() {
  cout << "Enter a street address, e.g. 36 Ocean View Road\n";
  getline(cin, input);
  i = 0;
  num = 0;
  while (input[i] != ' ') {
    num = num * 10 + input[i] - 48;
    i++;
  }
  if (num > 500) {
    cout << "This address is greater than 500";
  } else {
    cout << "This address is less than or equal to 500";
  }
}
```

Interesting things to notice in this program:

- the loop continues until a space has been found – the space after the number
- use the formula to convert characters to numbers – useful to remember this
- there are many ways to write a program – there will be alternatives

Concatenating strings

Concatenating two strings means placing the strings side by side to create a larger string. For example, if string1 is "Hello" and string2 is "there" and we concatenate them, we would get "Hellothere". In C++ we use the plus sign to do concatenation of strings.

```
#include <iostream>
#include <string>
using namespace std;

string string1, string2, string3;

int main() {
  string1 = "How are";
  string2 = "you today?";
  string3 = string1 + string2;
  cout << string3 << endl;
}
```

Remember:

With numbers, the plus sign is used for **addition**.
With strings, the plus sign is used for **concatenation**.

Example : reverse a string

A program to reverse a string:

```
#include <iostream>
#include <string>
using namespace std;

int i, total;
string sentence, reverse;

int main() {
  cout << "Type in a sentence ";
  getline(cin, sentence);
  total = sentence.length();
  reverse = "";
  for (i = total - 1; i >= 0; i--) {   // note the i--
    reverse = reverse + sentence[i];
  }
  cout << "The sentence is " << sentence << endl;
  cout << "The reverse is  " << reverse << endl;
}
```

Does this program work correctly? Do a ***walkthrough*** to find out.

Here is an example of a walkthrough for this program. Make a list of the variables and then work through the program, one instruction at a time. When a variable changes value, cross it out and write the new value next to the variable name.

```
i = 5  4  3  2  1  0  -1

j = 0  1  2  3  4  5  6

total = 6

sentence = My cat

reverse = tac yM
```

We also keep track of what is displayed on the screen, as follows:

```
Type in a sentence My cat
The sentence is My cat
The reverse is  tac yM
```

Example: replace a word

Replace a word in a sentence. Here is an example when the program is running:

```
Enter the word to find: bird
Enter the replacement word: fish
Enter a sentence: I saw a bird go by
```

```
Final sentence is: I saw a fish go by
```

And here is the program:

```
#include <iostream>
#include <cstdio>
#include <string>
using namespace std;

string findword, replace, sentence, final, word;
int total, i;

int main() {
  cout << "Enter the word to find: ";
  cin >> findword;
  getchar();
  cout << "Enter the replacement word: ";
  cin >> replace;
  getchar();
  cout << "Enter a sentence: ";
  getline(cin, sentence);
  total = sentence.length();
  final = "";
  word = "";
  for (i = 0; i < total; i++) {
    if (sentence[i] == ' ') {
      if (word == findword) {
        final = final + replace + " ";
      } else {
        final = final + word + " ";
      }
      word = "";
    } else {
      word = word + sentence[i];
    }
  }
  if (word == findword) {
    final = final + replace + " ";
  } else {
    final = final + word + " ";
  }
  cout << "Final sentence is: " << final << endl;
}
```

Interesting things to notice in this program:

- note the useful function `length()` that tells you how many characters in the string
- build up a word by looking for spaces in the sentence
- build up the final sentence by concatenating words (or replacement words)
- problem – it only works if there are spaces between words and not other characters
- note how you have to check for the final word after the loop has finished

Notes for C++

Section 9 : More on Strings

String functions

You can write your own functions that manipulate strings. Strings can be passed as parameters to functions and strings can be returned from functions.

Example: check for valid input

It is often useful to check that the input to a program is valid. For example, if a program needs to read an integer value, it would be good to check that the user has actually entered an integer value.

For example, read a number and then display this type of message:

645 is an integer value
Hello is not an integer value
6$4?5 is not an integer value

We can do this by first reading the input as a string and then checking the string to see if it has invalid characters. This is done in the program below (for integer values only):

```
#include <iostream>
#include <string>
using namespace std;

int stringtoint(string str, int total);

int value, total, i;
string temp;
bool isnumber;

int main() {
  cout << "Enter an integer value ";
  cin >> temp;
  isnumber = true;
  total = temp.length();
  for (i = 0; i < total; i++) {
    if ((temp[i] < '0') || (temp[i] > '9')) {
      isnumber = false;
    }
  }
  if (isnumber) {
    value = stringtoint(temp, total);
    cout << "The integer value is " << value << endl;
  } else {
    cout << temp << " is not an integer value!\n";
  }
}

int stringtoint(string str, int total) {
```

```
// function to convert a string to an integer
  int num, i;
  num = 0;
  for (i = 0; i < total; i++) {
    num = num * 10 + str[i] - 48;
  }
  return num;
}
```

Interesting things to notice in this program:

- read the number as a string and then check for any illegal characters
- what about negative numbers like -24? Will this function work for them?
- can you write a similar function to input real (float) numbers?
- `stringtoint` has a string parameter and returns an integer value
- `stringtoint` is a useful function to convert a string to an integer

Example: a function with string parameters

Write a program that reads a name (with a one-word first name and a last name) and then inserts a middle name between the first name and the last name. Here is an example run of the program:

```
Enter first name and last name separated by a space: Bilbo Baggins
Enter a middle name: Fred
The final name is: Bilbo Fred Baggins
```

We will write a string function to insert the middle name in the right place.

```
#include <iostream>
#include <string>
using namespace std;

string insert(string str1, string str2);

int main() {
  string firstlast, middle, full;
  cout << "Enter first name and last name separated by a space: ";
  getline(cin, firstlast);
  cout << "Enter a middle name: ";
  cin >> middle;
  full = insert(firstlast, middle);
  cout << "The final name is: " << full << endl;
}

string insert(string str1, string str2) {
// insert str2 into str1 where there is a blank character
  string temp;
  int total, i;
  temp = "";
  total = str1.length();
  i = 0;
  while (str1[i] != ' ') {
    temp = temp + str1[i];
    i++;
  }
```

```
  temp = temp + " " + str2;
  while (i < total) {
    temp = temp + str1[i];
    i++;
  }
  return temp;
}
```

The algorithm for the `insert` function:

- we will build up the full name in the string called temp
- copy the first part of the name (before the space) into temp
- concatenate the middle name onto temp (which is the first name)
- copy the second part of the name (after the space) into temp
- return temp back to main

Note that the function `insert` does not use any global variables. It is a completely "stand-alone" function. This means that it is a good candidate to put in a library for use by other programmers. Actually, there are already a number of insert functions available in various libraries.

Arrays of strings

We use arrays to store many data items of the same type. We can use an array of strings to store many strings.

Example: a program to store 4 names

```
#include <iostream>
#include <string>
using namespace std;

string str[4];
int i;

int main() {
  cout << "Enter the first name ";
  getline(cin, str[0]);
  cout << "Enter the second name ";
  getline(cin, str[1]);
  cout << "Enter the third name ";
  getline(cin, str[2]);
  cout << "Enter the fourth name ";
  getline(cin, str[3]);
  for (i = 0; i < 4; i++) {
    cout << "Name [" << i << "] is " << str[i] << endl;
  }
}
```

Interesting things to notice in this program:

- how would you modify the program to store 1000 names? Do not actually do this!

Looking at individual characters in an array of strings

Sometimes we want to look at individual characters in an array of strings. For example, load the names of three New Zealand towns and display only the first letter.

Example: a program to display the first letter of a name

```
#include <iostream>
using namespace std;

string town[55];   // the maximum number of towns is 55
int total, i;

int main() {
  total = 3;    // we are looking only at 3 towns
  town[0] = "Auckland";
  town[1] = "Wellington";
  town[2] = "Christchurch";
  cout << "The first letters of the towns are:\n";
  for (i = 0; i < total; i++) {
    cout << "The first letter of " << town[i];
    cout << " is " << town[i][0] << endl;
  }
}
```

Interesting things to notice in this program:

- we could have 55 towns (the maximum) but we only have 3 (the current total)
- the code `town[i][0]` means look at character `0` in string `i`
- you can check any character in a string but be careful not to go out of bounds

2D Arrays – this is NOT an important part of the course

The ***dimension*** of an array tells us how many index numbers are required to reference an element of the array. The arrays that we usually use are one-dimensional (1D), for example:

```
float rain[12];   // this is a 1D array
```

To work with an element of this array, we need one index number, for example: `rain[i] = 9.1;`

You can set up a two-dimensional array (2D), for example:

```
char chessboard[8][8];   // this is a 2D array
```

To work with an element of this array, we need two index numbers, for example:

```
chessboard[6][2] = 'P';
```

An array of strings is actually a 2D array of characters. We use two index numbers – the first index tells us which string we are working with and the second index tells us which character we are working with. For example:

```
town[i][0]
```

Problems with ASCII codes and alphabetical order

People like to put strings (especially names) into alphabetical order. For example, many people store names and numbers on their phones in alphabetical order. Why do people like alphabetical order?

We would like to write programs that store strings in alphabetical order. Unfortunately this is harder than it would appear because computers do not work with alphabetical order – they work with ASCII code order. Computers assume that a string is "greater than" (or "less than") another string based on the ASCII codes of the characters in order from the left.

For example, if we have two strings APE and APPLE, the computer would use the following process to check which of these strings is "greater than" the other one.

Set index to zero
Check character[index] of the first string against character[index] of the second string
In this example, check A against A – they are the same – add 1 to the index and repeat...
Check P against P – they are the same – add 1 to the index and repeat...
Check E against P – code for P is greater than the code for E so APPLE is "greater than" APE

This all works fine and "alphabetically" we know APPLE comes after (is greater than) APE. But what if we compare elephant with ZEBRA? The code for e is greater than the code for Z so elephant comes after ZEBRA in ASCII code order --- but alphabetically elephant should come before ZEBRA.

WARNING – when sorting words (e.g. names) into alphabetical order, it will only work if all the words are the same case, i.e. all upper case or all lower case.

Sorting names

We have already looked at a sorting algorithm to sort numbers. We now use exactly the same ***algorithm*** (but not the same ***program***) to sort names into alphabetical order. We can use the same algorithm because alphabetical order (in a computer) is actually numeric order based on the ASCII codes. So alphabetical order (in a computer) is actually numeric order in disguise.

Example: a program to sort names

```
#include <iostream>
#include <string>
using namespace std;

int getnames();
void sortnames(int total);

// a MAXIMUM of 150 names - you can change this if you like
string name[150];

int main() {
  int i, total;
  total = getnames();
  sortnames(total);
  cout << "\nHere is the list in alphabetical order\n";
  for (i = 0; i < total; i++) {
    cout << name[i] << endl;
  }
}

int getnames() {
  int i;
```

```
  i = 0;
  cout << "Enter names - type ??? to stop\n";
  getline(cin, name[i]);
  while (name[i] != "???") {
    i++;
    getline(cin, name[i]);
  }
  return i;
}

void sortnames(int total) {
  int pass, i;
  string temp;
  for (pass = 0; pass < total; pass++) {
    for (i = 0; i < total - 1; i++) {
      if (name[i] > name[i+1]) {
        temp = name[i];
        name[i] = name[i+1];
        name[i+1] = temp;
      }
    }
  }
}
```

Interesting things to notice in this program:

- main is short and readable thanks to functions – use functions to help ***design*** your program
- note how the sorting code is basically the same as the code for sorting numbers
- I made `name` a ***global variable*** – why? Could you make it a ***local variable***?
- My program works for a MAXIMUM of 150 names. What if you want more names?
- ??? is a ***trailing value*** for names. What if someone has the name ???
- Each function has a local variable called i. These variables all work separately.

Notes for C++

Section 10 : Text Files

Using files

Data in variables in a program is stored in the computer memory (RAM). However RAM is **volatile** – i.e. the data in it disappears when the power is switched off (or fails). If we want to keep data permanently we need to store it on a **storage device**. Storage devices include hard discs, optical storage devices (DVD) and flash memory. Many hard discs are now Solid State Drives (SSD) which is actually a type of flash memory. Devices have different access speeds: RAM is fastest, then flash memory, then hard discs, then optical devices. It will take longer to read data from a storage device than from RAM. For this reason, we often transfer data to RAM (put it into variables) before we use it. A device (such as a hard disc) often appears faster than it actually is because data is transferred to memory and used from there. While this increases speed it is important to realise that data is not actually on the disc, but is in volatile memory.

We store data in **files**. We write a program to load the data into the file (this is called ***writing*** to the file). Alternatively we can use other programs (e.g. an editor) to write data to the file. We can then write other programs to look at the data in the file (this is called ***reading*** the file). The data in the file is permanent and can be used many times after the file has been set up. Every file has a name. The name can include the drive, the path, the file name and the extension.

For example: `F:\University\159101\Program1.cpp` (use / in Linux)

Let us look at the various parts of the file name:

`F:` this is the **drive letter**. Each device has a letter, e.g. the hard disc on a home PC is usually C: Different **partitions** on a hard disc may also have different drive letters. If you are using a file on a storage device (e.g. flash memory) ensure that you have the correct drive letter for that device. What drives do the letters A: and B: stand for? More information about drive letters can be found at http://en.wikipedia.org/wiki/Drive_letter_assignment

`\University\159101\` this is the **path** (or the list of folders). In this example, there is a folder called `University` and inside that folder there is another folder called `159101`.

`Program1` this is the **file name**. Use meaningful names so that you just have to look at the name and it tells you what is stored inside the file. File names must not contain strange characters such as a dot or a comma or similar.

`.cpp` this is the **extension**. The extension tells us the type of the file, e.g. this file is a C++ program. Examples of extensions include: `.docx .txt .jpg .cpp .java .xlsx .exe` People keep inventing extensions – that is why there are such a lot of them!

There is a short cut that we often use that avoids all these complications with file names. When using a file from inside a program, you do not have to supply the drive letter and the path as long as the file is in the same directory (or folder) that you are working in. For example, if we were using the file given above and were working inside the same folder we could then use the name `Program1.c` and we would not have to provide the drive letter and the path.

Unfortunately, certain operating systems (especially Windows) try to hide the file extensions. Always change the folder options so that you can see the extensions as they are important information. In

159.101 we will be using the extension `.txt` because we will mainly read and write text files. However text files can have other extensions and, in fact, you can invent your own extensions!

Text files

There are many different kinds of files. In our course we will use only at text files. **Text files consist of lines of simple text.** There are no fonts (e.g. italics) in a text file. Any program you have written is a text file as it consists of lines of text. However, jpg files or Word files are not text files as are encoded in a particular way. See what happens in you open a jpg file with and editor such as Wordpad. Note that text files may have various extensions, e.g. .cpp and .txt are both extensions that are used for text files.

Using text files from a C++ program

The basic rules for using a file from your own program are:

- Know the full name of the file
- Open the file
- Access the file (reading or writing)
- Close the file

A light-hearted reminder of this process can be found here:
https://www.youtube.com/watch?v=-zQgEqDdvuo

Remember that you can always read or write to a text file using any editor such as Wordpad, etc.

A useful cheat about entering file names

As describes above, a file name can be rather lengthy and have many sections – the drive letter, the path, etc. Sometimes it is necessary to enter the FULL file name as discussed in this section. However, after a while you will notice that our examples use much simpler file names such as Names.txt. The drive letter, the path, etc are all missing – we use only the file name and the extension.

This shortcut works ONLY IF the file is in the same folder as your program. As long as you put the data file in the same folder as the program, then you can leave out the drive letter and the path.

When entering a file name, you have a choice:

a) Enter the FULL file name including drive letter, path, etc
b) Enter only the file name and extension BUT the data file must be in the same folder as the program.

A program to load lines of text into a text file (writing to the file)

This program demonstrates how to create a new file and load data into the file – known as ***writing*** to the file. We will load four lines of text into the file.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
fstream datafile;          // define a file object
string filename, text;

int main() {
  // 1. Load the full name of the file into a string
  filename = "F:\\TESTING\\data.txt";   /* use \\ for \ */

  // 2. Open the file for writing (output)
  datafile.open(filename.c_str(), fstream::out);

  // 3. Access the file - in this case, write to the file
  cout << "Now writing to " << filename << endl;
  text = "This is the first line in the file";
  datafile << text << endl;
  text = "This is the second line in the file";
  datafile << text << endl;
  text = "And another line in the file...";
  datafile << text << endl;
  text = "This is the last line in the file";
  datafile << text << endl;
  cout << "Completed writing to file.\n";

  // 4. Close the file
  datafile.close();
}
```

We have introduced several new concepts in this program. Here is an overview:

`#include <fstream>`
We include the library `fstream` when working with files. The library `iostream` provides functions to move data to and from IO (input-output) devices and `fstream` provides functions to move data to and from files.

`fstream datafile;`
This line defines a file object and gives it a name. You can use any name (similar to variable names). `datafile` is an object (more on objects later) and is used to let the system know which file you are working with.

`filename = "F:\\TESTING\\data.txt";`
Load the name of the file into a string variable. Note that you need \\ (special character) to indicate a single backslash. **WARNING:** this program will only run if the USB is drive F: and there is already a folder called TESTING on the USB. Set this up before running the program. The easiest way to sort out the file name is to **have only the file name and extension** (no drive letter and no path) and then the text file will appear in the folder with your program.

`datafile.open(filename.c_str(), fstream::out);`
Open the file for writing (output). This line connects the file object (datafile) with the file name. The first parameter must be the "C string version" of the filename, which is why we show the filename as `filename.c_str()`. The second parameter informs the system if the file is used for writing (output) or reading (input) or something else – there are some other alternatives.

`cout << "Now writing to " << filename << endl;`

This line shows a message on the screen. This is not needed but many people get anxious if the run an application and it shows a blank screen. When designing a program that would otherwise have a blank screen, it is a good idea to display a message to let people know what is going on.

`datafile << text << endl;`
This line loads text to the file using the file object. You write the instruction following all the rules for `cout` but you use the name of the file object instead of `cout`.

`datafile.close();`
This line closes the file. The file object `datafile` cannot be used again in the program after this function call.

This program will ***create a new file***. Before you run the program there is not a file called `data.txt`. If the program runs successfully, the file will appear.

IMPORTANT – after running a program to load data into a file, you should always check that the file has appeared. It may be on a USB drive (depends on the drive letter). Also look at the file itself to see if the lines of text were output correctly. You can look at a file by using and text editor such as Wordpad, etc.

PRACTICE – you must practice writing text to files. One exercise is to select an old (simple) program and convert it to output line to a file instead of to `cout`.

A program to append lines of text at the end of a text file (appending)

Files often need to be edited. We will take the file we created in the previous program and we will include two new lines at the end of the file – adding new lines at the end is called **appending**.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream myfile;              // define a file object
string filename, text;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for both output and appending
  myfile.open(filename.c_str(), fstream::out | fstream::app);

  // 3. Access the file - in this case, write to the file
  cout << "Now appending lines to " << filename << endl;
  text = "This is a new line to be appended";
  myfile << text << endl;
  text = "This is another new line to be appended";
  myfile << text << endl;
  cout << "Completed writing to file.\n";

  // 4. Close the file
```

```
  myfile.close();
}
```

Notes about this program:

`fstream myfile;`
In this program I decided to use the name `myfile` for the file object. You can use whatever name you want to but get into the good habit of using meaningful names.

`// 1. The user will enter the full name of the file`
In this program, the user has to enter the file name. There are two things the user has to get right:
a) make sure you know the EXACT file name before entering it
b) do not use the \\ when entering data (only use \\ in the code). So in this case the correct file name to be entered would be `F:\TESTING\data.txt`

`myfile.open(filename.c_str(), fstream::out | fstream::app);`
The file is opened for both output and appending. Because you have informed the system that you are appending, any output to the file will be added after the existing lines in the file.

After the program has run, **CHECK THE FILE**. Use an editor to open the file and check the the new lines have been appended at the end.

A program to load lines of text from a file (reading from the file)

The previous example covered loading data from a program into a text file (writing). We will now move on to loading data from a text file into a program (reading).

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int number;
fstream inputfile;          // define a file object
string filename, line;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  inputfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (inputfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {

    // 3. Access the file - read all the lines from the file
    cout << "Displaying all the lines from " << filename << endl;
    number = 0;
    while (getline(inputfile, line)) {
```

```
      number++;
      cout << number << ": " << line << endl;
    }
    cout << "Displayed " << number << " lines from the file.\n";

    // 4. Close the file
    inputfile.close();
  }
}
```

Notes about this program:

`// 1. The user will enter the full name of the file`
You can use this program to read any file you like. Before entering the file name:
a) make sure you know the EXACT file name before entering it
b) make sure the USB is plugged in (and what drive letter it is) – if you are using the USB
c) do not use the \\ when entering data (only use \\ in the code), e.g. `F:\TESTING\data.txt`

`inputfile.open(filename.c_str(), fstream::in);`
Open the file for reading (input).

`if (inputfile.is_open() == false) {`
The opening of the file may fail for a variety of reasons, e.g. you may have entered the incorrect name, the USB may not be plugged in, you included an incorrect folder name in the path, etc. It is important to use the `is_open` function to check if a file has opened successfully. If it has not opened, print an error message alerting the user to that fact that the file could not be opened. This check is only done when reading from a file. When you are writing to a file, a new file will be created anyway.

`while (getline(inputfile, line)) {`
We set up a loop to read every line from the file. It is a while loop because we do not know how many lines there will be. We have already used `getline` with `cin`, now we use it with the file object.

`number++;`
I included a line number variable to count how many lines I had read from the file. This is entirely optional and you do not have to do this unless you actually need the line numbers.

Reading one word at a time from a text file

Sometimes we want to read one word at a time from a text file. Reasons for doing this include:

a) counting words – students are often told "write an essay of 4,000 words" or similar. It is useful to write programs that read in the text file and counts the number of words in the file. People make money by selling programs to count words – google "word count" to find out more!

b) analysing text – people study books and documents to look for patterns, word use, etc. Once again, google "analyse text" to find out more about this.

c) compiling computer programs – a program written in C (or any other language) cannot be executed on a computer. It first needs to be converted to machine code. This process is carried out by a compiler – a program that reads other programs and checks every instruction.

A program to read one word at a time from a text file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int wordcount;
fstream datafile;          // define a file object
string filename, word;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  datafile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (datafile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {

    // 3. Access the file - read one word at a time from the file
    cout << "Displaying all the words from " << filename << endl;
    wordcount = 0;
    while (datafile >> word) {
      wordcount++;
      cout << word << endl;
    }
    cout << "There are " << wordcount << " words in the file.\n";

    // 4. Close the file
    datafile.close();
  }
}
```

Interesting things to notice in this program:

- as usual, only run the program if the datafile has opened successfully
- use `datafile >> word` to read one word at a time from the file
- the while loop will keep going until the file runs out
- look at the display – are these words actually "words" or not...? Note that the punctuation (quotes, semi-colons, etc) all appear as part of the words.

A program to count if-statements in another program

A compiler is a program that reads in another program and checks every statement.
We are not going to write a compiler but it is interesting to get an idea of what a compiler does.
Write a program to read in another program and look for the if statements in the other program.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
int ifcount;
fstream programfile;          // define a file object
string filename, str;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the program: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  programfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (programfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {

    // 3. Access the file - read one word at a time from the file
    ifcount = 0;
    while (programfile >> str) {
      if (str == "if") {
        ifcount++;
      }
    }
    cout << "There are " << ifcount << " if statements.\n";

    // 4. Close the file
    programfile.close();
  }
}
```

WARNING: this program has a ***logic error*** – it runs but sometimes produces an incorrect count. For example, if you run it and look through the program listed above (i.e. the program looks at itself!) it will count 4 if statements but there are actually only 2 if statements. Can you see why?

A logic error is the worst kind of error because the program sometimes produces the correct result and sometimes not. The only way to find a logic error is by extensive testing.

Challenge 1: fix the program to accurately count if statements

Challenge 2: modify the program to display line numbers, i.e. for each if statement it should display the line number of that particular if statement.

Read one LINE at a time or one WORD at a time?

Creating a program is all about making decisions. For example, when to use a while loop and when to use a for loop? Should I use a function or not? Should I use local variables or not? Similarly, when reading data from a file you need to decide if you are going to read **one line at a time** or **one word at a time**.

Saving lines of data using a standard template

Text files can be used to save data where each line in the file represents a data entry and a standard pattern (or template) is used for each line.

For example, we may want to save information about books and we could define the template to be:
Title;Author;Year

An example of one entry (for one book) in our file could be a line that looks like this:
The Two Towers;Tolkien J.R.R.;1954

Each line in the file saves the title of the book, the author of the book, and the year in which the book was first published. You could easily expand the system to save more information if you want to.

Note that each item on the line is separated by a semi-colon. These are called ***separators*** or ***delimiters***. A separator can be anything you like so long as it does not normally appear in the title or author of a book. Finding a good separator is similar to finding a good trailing value – it must not be legal data.

We can now write a program that reads data from the file and checks the data in some way. Examples of typical checks could be – how many books are there in the file? - how many books were published before 1999? - how many books are written by a particular author? - how many books have the word "computer" in the title? etc, etc. There are lots of possible checks that can be done.

How do we approach this program? Should we read one line at a time or one word at a time?
- reading one line at a time means we don't separate the title, author and year
- reading one word at a time means we get "words" such as "Towers;Tolkien"

We will take the approach of reading one line at a time and then breaking the line into the separate sections. This does not mean this approach is better – it is just one possible alternative.

A program to find books written by a particular author

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void breakstring(string s);

int count;
fstream bookfile;          // define a file object
string filename, searchname, line, title, author, yearstring;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;
  getchar();  // clear the input buffer before reading the author

  // 2. Open the file for reading (input)
  bookfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (bookfile.is_open() == false) {
```

```
    cout << "ERROR: not able to open " << filename << endl;
  } else {
    // 3. Enter the name of the author you want
    cout << "Enter the name of the author: ";
    getline (cin, searchname);   // names can have spaces
    cout << "Here is the list of books written by ";
    cout << searchname << endl;

    // 4. Access the file - read all the lines from the file
    count = 0;
    while (getline(bookfile, line)) {
      cout << line << endl;  // DEBUG cout to display all lines
      breakstring(line);
      // convert yearstring to a number
      if (author == searchname) {
        cout << yearstring << " " << title << endl;
        count++;
      }
    }
    cout << searchname << " wrote " << count << " books.\n";

    // 5. Close the file
    bookfile.close();
  }
}

void breakstring(string s) {
  // break the string s into smaller strings using the separator
  int total, i;

  i = 0;
  title = "";
  while (s[i] != ';') {
    title = title + s[i];
    i++;
  }
  i++;  // jump over the semi-colon
  author = "";
  while (s[i] != ';') {
    author = author + s[i];
    i++;
  }
  i++;  // jump over the semi-colon
  yearstring = "";
  total = s.length();
  while (i < total) {
    yearstring = yearstring + s[i];
    i++;
  }
}
```

Interesting things to notice in this program:

- before running the program, make a text file with some lines that follow the pattern
- we need to clear the input buffer before reading the name of the author

- the author name must be read as a line of text because names often have spaces in them
- it is useful to display lines as you read them to check that the file is correct – you can remove this debug cout when the program is working.
- the breakstring function is used to help with program design – it keeps main shorter
- the function is not "stand alone". It uses strings that are global variables because a function can only return one answer and we want 3 answers (the 3 short strings).
- yearstring is a string. It is not a number. You need to convert it to a number if, for example, you wanted to check which books were published before 1999.
- are there other ways to write this program? Definitely, but it is good to get used to the fact that strings are arrays of characters.
- note the extra i++ in the function to jump over the semi-colons.
- the last loop in the function does not look for a semi-colon. It runs to the end of the string.
- the function has a logic error. It works fine most of the time but there is one situation when it displays the wrong information. What situation is this and how would you fix it?

Can we access more than one file in a program?

Yes we can. You can open as many files as you like. Make sure you have a file variable for each file that you are using and that you close all the files. Here is an example...

Program to read names from a file and separate A – L from M – Z

There is a text file called Names.txt which has one name (perhaps with spaces) on each line of the file. The file is not sorted. We want to read all the names from the file and create two new files where one of the files contains the names from A to L and the other file contains the names from M to Z.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream fileone, filetwo, filethree;   // define three file objects
string filename, name;

int main() {
  // 1. Open the first file for reading
  filename = "Names.txt";
  fileone.open(filename.c_str(), fstream::in);

  // 2. Open the other two files for writing
  filename = "AtoL.txt";
  filetwo.open(filename.c_str(), fstream::out);
  filename = "MtoZ.txt";
  filethree.open(filename.c_str(), fstream::out);

  // only continue if the file has opened successfully
  if (fileone.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {
    // 3. Access the file - read all the lines from the file
    cout << "Processing the names to create new files...\n";
```

```
    while (getline(fileone, name)) {
      cout << name << endl;  // DEBUG cout to display all lines
      if ((name[0] >= 'A') && (name[0] <= 'L')) {
        filetwo << name << endl;
      } else {
        filethree << name << endl;
      }
    }
    cout << "File processing complete, check the new files.\n";

    // 4. Close all the files
    fileone.close();
    filetwo.close();
    filethree.close();
  }
}
```

Transferring data between your program and a standard file format

It is useful to be able to transfer data between our own programs and standard packages. Many packages are set up to be able to do this. For example, spreadsheets such as Excel are usually set up to read data from or write data to simple text files called 'comma-separated value' (csv) files. A csv file has a line of entries (numbers or strings) separated by commas.

Program to write data into a csv file

Students have written two tests. Read the student names and the two marks from the screen and write the data into a csv file. Then open the csv file with a spreadsheet.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream csvfile;          // define a file object
int mark1, mark2;
string filename, name;

int main() {
  // 1. open the file
  filename = "Test-results.csv";
  csvfile.open(filename.c_str(), fstream::out);

  // 2. enter the data using a trailing value of ?? for the name
  cout << "Enter the student name and end with ?? ";
  getline (cin, name);  // names may have spaces
  while (name != "??") {
    cout << "Enter the result for test 1: ";
    cin >> mark1;
    cout << "Enter the result for test 2: ";
    cin >> mark2;
    getchar();            // clear the input buffer
    csvfile << name << "," << mark1 << "," << mark2 << endl;
```

```
    cout << "Enter the student name and end with ?? ";
    getline (cin, name);  // names may have spaces
  }
  cout << "You have created the file called " << filename << endl;
  // 4. Close the file
  csvfile.close();
}
```

Two ways of opening a csv file

Option 1: Open the file as a spreadsheet. Usually this is done by double-clicking on the file (it must have the extension .csv). You will not see the commas in the spreadsheet.

Option 2: Open the file as a text file. Open the file through an editor such as gcc, notepad, etc. Run the editor first, then select open, then select the file. You will see the commas in the csv file.

Program to read data from a csv file

Make a spreadsheet where column 1 is an ID number and column 2 is an exam result, for example:
```
6847   53
```
Save the spreadsheet as a csv file. You do this by selecting "Save As" and then changing the type of the file to "CSV (Comma delimited) (*.csv)". There are several types of csv file and the other types may not work correctly.

Note that the numbers in the csv file are saved on one line with a comma between them. Write a program that reads from the csv file into corresponding arrays (ID and mark). Calculate the average and print all IDs where the exam result is higher than the average.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream csvfile;          // define a file object
int id[100], exam[100], count, i;
float average, total;
string filename;
char comma;

int main() {
  // 1. The user will enter the full name of the csv file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  csvfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (csvfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {
    // 3. Read the data one number at a time
    i = 0;
```

```
    total = 0.0;
    // read lines from the file - note how we read the comma
    while (csvfile >> id[i] >> comma >> exam[i]) {
      total = total + exam[i];
      i++;
    }
    // 4. Close the file
    csvfile.close();

    // 5. Calculate and display the results
    count = i;
    average = total / count;
    cout << "The class average is " << average << endl;
    for (i = 0; i < count; i++) {
      if (exam[i] > average) {
        cout << "Student " << id[i] << " achieved ";
        cout << exam[i] << endl;
      }
    }
  }
}
```

Things that can go wrong with this program

The biggest problem is not actually the program but the creation of the csv file. Depending on your version of Excel, and various other factors, you may get invisible characters embedded in your csv file. If this happens, the program will not produce the correct output. If this happens, use the version of the program below which reads each line as a string and then extracts the numbers.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// this version of the program reads lines of text
// and then converts to numbers

fstream csvfile;          // define a file object
int id[100], exam[100], count, num, i, k;
float average, total;
string filename, line;;

int main() {
  // 1. The user will enter the full name of the csv file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  csvfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (csvfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {
```

```
    // 3. Read lines of text and convert to numbers
    i = 0;
    total = 0.0;
    while (getline(csvfile, line)) {
      cout << line << endl; // debug statement
      line = line + "$";  // use a $ to mark the end of the line
      k = 0;
      num = 0;
      while (line[k] != ',') {
        num = num * 10 + (line[k] - 48);
        k++;
      }
      id[i] = num;
      k++;  // skip over the comma
      num = 0;
      while (line[k] != '$') {
        num = num * 10 + (line[k] - 48);
        k++;
      }
      exam[i] = num;
      total = total + exam[i];
      i++;
    }
    // 4. Close the file
    csvfile.close();

    // 5. Calculate and display the results
    count = i;
    average = total / count;
    cout << "The class average is " << average << endl;
    for (i = 0; i < count; i++) {
      if (exam[i] > average) {
        cout << "Student " << id[i] << " achieved ";
        cout << exam[i] << endl;
      }
    }
  }
}
```

Notes for C++

Section 11 : Classes and Objects

A **class** is a collection of variables (**data)** and **methods** (functions). Usually the data in the classes is part of a logical collection – sometimes called a **record**. For example, a student record may consist of student name, student ID, student email, degree, completed courses, etc.

Example A – setting up a simple date class

```
#include <iostream>
using namespace std;

class date_class {
public:
  int day, month, year;
};

date_class today;

int main() {
  today.day = 23;
  today.month = 11;
  today.year = 2019;
  cout << today.day << "/" << today.month;
  cout << "/" << today.year << endl;
}
```

Important points about this program:

- we set up a **class** called `date_class` with three data items which are day, month, year
- **public** means that the data items can be used anywhere in the program (e.g. inside main)
- we use the class to define an **object** called `today`. One class can be used to create many objects. Sometimes a class is called a **template**.
- we refer to data items inside an object with a dot, for example `today.month`
- data items are normal variables and can be used in the same way as variables

Methods

Classes not only hold variables, they also provide functions to work with the data. These functions are called methods. The methods serve two purposes:

a) they make main shorter and easier to read
b) they make the classes portable (or stand-alone) so they can be placed in a library

Example B – adding methods to the date class

We add two methods to the class – one is to set a date and the other is to display a date.

```
#include <iostream>
using namespace std;

class date_class {
public:
```

```
  int day, month, year;
  void setdate(int d, int m, int y);
  void displaydate();
};

date_class today;

int main() {
  int m;
  today.setdate(23, 11, 2019);
  today.displaydate();
  m = today.month;
  cout << "The month in this date is " << m << endl;
}

void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
}
```

Important points about this program:

- sections of main are shorter than Example A (for example, setting the date)
- each method has a **function prototype** inside the class
- Example A shows us that methods are optional (but are often used)
- the methods are functions – they follow all the rules of functions
- the class variables are used inside the methods without a dot, e.g. `day`
- the methods are called with the object name and a dot, e.g. `today.displaydate();`

Methods are **functions** so we follow all the rules about functions. All rules about **parameters** and the use of **return** apply as usual. Each method (function) has a **prototype** but the prototypes sit inside the class. Each method (function) has a **definition** (the code for the function) and these are placed, as usual, after main. However, we need to warn the system that these functions are not simply ordinary functions, they are methods that belong to a particular class. So we write the name of the class before the name of the function with the symbol `::` which means "belongs to". For example, the method `setdate` belongs to the class `date_class` and so we write `void date_class::setdate`

Public and Private

Data and methods inside a class can be defined to be public or private. Private data and methods can only be used within the class itself – they may not be used by main (or any other function). Public data and methods can be used anywhere in the program (inside the class, main, other functions, etc).

You can decide which data and which methods are private and which are public. We will follow a widely-used convention which is:

- make all data private
- make all methods public

The reason for this is that it enables classes to become portable (i.e. can be held in a library and used by many different programs). This would not be possible if the main program had to know about the data in order to use the class.

Example A (above) is ***not*** portable – main has to know all details of the class in order to work
Example B (above) is portable – main does not have to know about the data items in the class

Example C – adding methods to the date class

```
#include <iostream>
using namespace std;

class date_class {
private:
  int day, month, year;
public:
  void setdate(int d, int m, int y);
  int getday();
  int getmonth();
  int getyear();
  void displaydate();
};

date_class today;

int main() {
  int m;
  today.setdate(23, 11, 2019);
  today.displaydate();
  m = today.getmonth();
// m = today.month;  -- causes an error because month is private
  cout << "The month in this date is " << m << endl;
}

void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

int date_class::getday() {
  return day;
}

int date_class::getmonth() {
  return month;
}

int date_class::getyear() {
  return year;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
```

```
}
```

Important points about this program:

- it does the same thing as Example B
- main is not allowed to use the private data that belongs to the class
- main can use the methods which are public
- we have added several "get" methods to the class

Set and Get methods

It is very common to see "set" and "get" methods inside a class. For example, in the program above, `setdate` is used to load the variables and `getday` is used to send the variable `day` back to main. The reason for using "set" and "get" methods is that the data in the class is private and cannot be directly accessed by main. Instructions such as `m = today.month;` are no longer allowed because main is not allowed to directly access the private data. Instead we use `m = today.getmonth();` to send the value of the private variable `month` back to main.

Programmers often have lengthy discussions of which approach is most sensible – is it better to make all data private and then set up "set" and "get" methods? This is the conventional approach but can produce enormously long code if there are many private variables in the class and every variable needs a "set" method and a "get" method. The alternative approach is to make the class data public. This means that main can directly access the class data (see Example A above) but can lead to problems when updating a class, especially if that class is in a library and is used by many programs (more on updating classes later). We will stick to the conventional approach which is:

a) make all class data private and all methods public
b) write "set" and "get" methods as required

Short methods can be defined inside the class

Some methods are very short with only 1 or 2 short lines of code. For example, many "set" and "get" methods are very short. It is a convention that, when a method is very short, it is written directly into the class. In other words, the prototype is replaced by the function definition. It then does not appear in the list of functions (and methods) after main. A method that is defined inside the class is usually written all on one line including the curly brackets.

Some points to remember:

- the method definitions can be written EITHER inside the class OR after main
- if the method definition is inside the class then it replaces the prototype
- any method can be written inside the class but by convention we do not do this for methods that have many lines of code – usually only methods with 1 or 2 lines of code appear in the class

All this is optional. You don't have to do it but you need to be able to recognise it in other people's programs. The only reason for placing method definitions directly into the class is that it makes the overall code shorter.

Classes inside classes

We can define several classes in one program and classes can make use of other classes. For example, we may like to keep a list of people's birthdays where each entry on the list has a name and a birthday and each birthday is a date.

Example D – using classes inside classes

```
#include <iostream>
using namespace std;

class date_class {
private:
  int day, month, year;
public:
  void setdate(int d, int m, int y);
  int getday() { return day; }
  int getmonth() { return month; }
  int getyear() { return year; }
  void displaydate();
};

class birthday_class {
private:
  string name;
  date_class birthday;
public:
  void loaddata(string n, int d, int m, int y);
  void display();
  bool compare(birthday_class other);
};

birthday_class birthday1, birthday2;

int main() {
  birthday1.loaddata("Madonna", 16, 8, 1958);
  birthday1.display();
  birthday2.loaddata("John Tolkien", 3, 1, 1892);
  birthday2.display();
  if (birthday1.compare(birthday2) == true) {
    cout << "They have the same birthday\n";
  } else {
    cout << "They do not have the same birthday\n";
  }
}

//--------------- methods for the date_class ------------
void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
}

//--------------- methods for the birthday_class ------------
void birthday_class::loaddata(string n, int d, int m, int y) {
  name = n;
  birthday.setdate(d, m, y);
```

```
}

void birthday_class::display() {
  cout << name << " birthday on ";
  birthday.displaydate();
}

bool birthday_class::compare(birthday_class other) {
  int day1, day2, month1, month2;
// people have the same birthday if day and month are the same
  day1 = birthday.getday();
  month1 = birthday.getmonth();
  day2 = other.birthday.getday();
  month2 = other.birthday.getmonth();
  if ((day1 == day2) && (month1 == month2)) {
    return true;
  }
  return false;
}
```

Important points about this program:

- short methods are defined directly in the class – written as one line
- there are two classes and the `birthday_class` makes use of the `date_class`
- the private rules are still followed, so the birthday_class has to use `birthday.getday()`
- main does not use the `date_class` at all
- note the use of comments to make headings for the method definitions of each class

Arrays of objects

We can set up arrays of objects. For example, write a program that displays all the days that were holidays in New Zealand in a particular year.

Example E – arrays of objects

```
#include <iostream>
using namespace std;

class date_class {
private:
  int day, month, year;
public:
  void setdate(int d, int m, int y);
  void loaddate();
  int getday() { return day; }
  int getmonth() { return month; }
  int getyear() { return year; }
  void displaydate();
};

date_class holiday[20];  // an array of objects
int i, num, year;
```

```
int main() {
  cout << "Enter the year ";
  cin >> year;
  cout << "Enter how many holidays in " << year << ": ";
  cin >> num;
  cout << "Enter all the dates that are holidays in ";
  cout << year << endl;
  for (i = 0; i < num; i++) {
    holiday[i].loaddate();
  }
  cout << "\nList of NZ holidays in " << year << endl;
  for (i = 0; i < num; i++) {
    holiday[i].displaydate();
  }
}

//--------------- methods for the date_class ------------
void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

void date_class::loaddate() {
  cout << "Enter the date (for example: 18 4 2019) ";
  cin >> day >> month >> year;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
}
```

Important points about this program:

- set up an array of objects in the usual way by using the square brackets
- all rules of arrays apply, e.g. index numbers, out of bounds, etc
- note how the array uses the methods, e.g. `holiday[i].loaddate();`

Strings in C++ are objects

When we create a string in C++, we are actually creating an object. For example:
`string msg;`
There is a class called string and we are creating an object called `msg`.

The string class has several methods such as length(). Another interesting method is capacity(). This tells you how many characters can fit into the string. If you load a string with more characters than the capacity, the string will automatically expand to fit the new length of the string.

For example, the string is changed to be longer in the following program. Note how the capacity expands to fit the new string.

```
#include <iostream>
#include <string>
using namespace std;
```

```
string msg;
int len, cap;

int main() {
  msg = "Hello";
  cout << msg << endl;
  len = msg.length();
  cout << "len = " << len << endl;
  cap = msg.capacity();
  cout << "capacity = " << cap << endl;

  msg = "My name is Bilbo Baggins";
  cout << msg << endl;
  len = msg.length();
  cout << "len = " << len << endl;
  cap = msg.capacity();
  cout << "capacity = " << cap << endl;
}
```

Most of the time, it is really useful that a string just expands when needed. However, it comes at a cost. Every time a string has to expand its capacity, the system will spend time doing this. In a time-critical operation, this may cause a problem.

There are a huge number of string methods available in the string class. For more information, see: http://www.cplusplus.com/reference/string/string/

In 159.101 you should be aware that a string in C++ is actually an object. You do not need to know the full string class or the methods (except for length – which we already know about).

File variables in C++ are objects

When we create a file variable in C++, we are actually creating an object. For example:
`fstream datafile;`
There is a class called fstream and we are creating an object called `datafile`.

The fstream class has a lot of private data that is required when accessing a file. For example, it keeps track of which line you have just read from a file and which line will be the next one to be read.

We already know several methods from the fstream class such as:
`datafile.open`
`datafile.close`
`datafile.is_open`

There are many methods available in the fstream class. For more information, see:
http://www.cplusplus.com/reference/fstream/fstream/

In 159.101 you should be aware that a file variable in C++ is actually an object of the fstream class. You do not need to know the full fstream class or all the methods – except for those methods that we have used in Section 10 of the notes.